

PART I · 基础

PART II · Skills

PART III · Subagents

PART IV · Hooks

PART V · 组合

PART VI + 附录

本 PART 章节

10. Ch 10 · Subagents Fundamentals

11. Ch 11 · Designing Effective Subagents

12. Ch 12 · Parallel Subagents and the Orchestrator Pattern

原书第 91 页 · PART THREE / CHAPTER TEN 起始

91

PART THREE · 第三部分

Subagents: Claude's Parallel Specialist System

子代理:Claude 的并行专员系统

One mind, many specialists. The work that does not need to share thinking should not have to share context.

"一心多用,各专一业。"——不需要共享思考的工作,本就不该被迫共享上下文。

CHAPTER TEN · 第十章

Subagents Fundamentals: Why Single-Threaded Claude Isn't Enough

子代理基础:为什么"单线程 Claude"远远不够

"Specialization is the natural answer to the limits of attention. The mind that tries to hold everything ends up holding nothing well."

— Adam Smith, paraphrased

"面对注意力的边界,专门化是自然应运而生的答案。一颗试图包揽一切的心,最终什么都握不好。"

—— 亚当·斯密(转述)

后一页 →

原书第 92 页 · CHAPTER TEN(续)

92

Skills, the subject of Part Two, taught Claude how to do specific kinds of work consistently. They are an answer to the question of capability. The chapters that follow are about a different question, and a different answer. The question is what to do when work that needs to happen in a single session crosses several different kinds of attention, when the context window starts filling with material that the current task does not

need, when the developer wants three things investigated at once instead of one, and when the cognitive cost of switching modes inside a single Claude session begins to degrade the quality of every output. The answer to these problems is the second pillar of the framework, the subagent.

第二部分讲的 Skills,教的是 Claude 如何稳定地做好某一类具体工作——它回答的是"能力"的问题。接下来的几章,问的是一个不同的问题,也给出一个不同的答案。当一个 session 里的工作横跨好几种"注意力类型"、当上下文窗口开始被与当前任务无关的内容塞满、当开发者希望三件事并行调研而不是一件一件来、当"在同一个 Claude session 里反复切换模式"所产生的认知代价开始让每一条输出都降级时,该怎么办?这些问题的答案,就是框架里的第二柱——subagent(子代理)。

Begin with a definition that will guide every chapter in this part of the book. A subagent is a separate Claude process, with its own system prompt, its own restricted set of tools, and its own isolated context window, that the main session can dispatch to perform a specific kind of work. When the subagent finishes its work, only its final output returns to the main session. The dozens of tool calls the subagent may have made along the way, the files it may have read, the drafts it may have considered, the dead ends it may have explored, none of that enters the main session. The main session sees a clean result, and the work that produced the result happens in a parallel space that the main session never has to see.

先给一个定义,后续每一章都会用到它。一个 subagent 是一个独立的 Claude 进程——它有自己的 system prompt、自己受限的工具集、自己隔离的上下文窗口——主 session 可以把它派出去,执行某一类特定的工作。subagent 完成工作之后,只有它的最终输出会回到主 session 之中:子代理一路调过的几十个工具、看过的文件、考虑过的草稿、走过的死胡同,这一切都不会进入主 session。主 session 看到的只是一份干净的结果,真正产出这份结果的工作,发生在主 session 永远不必看见的"平行空间"里。

This is a substantial architectural commitment. To understand why the commitment is worth making, it is worth examining what happens to a Claude Code session that does not use subagents and that is asked to do something complex. Take a real example. A developer asks Claude Code to add a new feature to a substantial codebase. The work involves understanding the existing architecture, designing the new addition, writing the code, writing tests for it, updating documentation, and reviewing the whole change for quality. A single Claude Code session can do all of these things in sequence, and it often does. But there is a hidden cost.

这是一项相当重的架构承诺。要理解这个承诺为什么值得付出,值得先看看:当一个 Claude Code session 不使用 subagent、却被要求做一件复杂的事时,会发生什么。举一个真实的例子:开发者让 Claude Code 给一个相当规模的代码库加一个新功能。这件工作包括:理解现有架构、设计新增内容、写代码、给它写测试、更新文档、再对整组改动做一遍质量评审。一个 Claude Code session 完全能按顺序把这些事做完,而且它经常就是这样做的。但这里有一个隐性的代价。

In the first phase, Claude reads many files to understand the existing architecture. Each file Claude reads adds to the context window. The context grows by hundreds or thousands of tokens for every file. By the time Claude has built a working understanding of the architecture, the context window is full of source code, much of it only loosely relevant to the new feature.

在第一阶段,Claude 要读很多文件,以理解现有架构。每读一个文件,上下文窗口就多一份内容——每多一个文件,就多几百到几千 token。等到 Claude 真的把架构"读到能上手"的时候,上下文窗口里已经堆满了源代码,而且其中相当一部分只是松散地和新功能相关。

← 前一页

后一页 →

In the second phase, Claude designs the new addition. The design involves several false starts, each of which leaves traces in the context. By the third phase, when Claude is writing the actual new code, the context window contains the architectural exploration, the design exploration, and the writing of the new code, all jumbled together. By the time Claude reaches the test-writing phase, the context window is dominated by historical material that has nothing to do with testing, and the test-writing suffers as a result, because the model is trying to write tests while looking at the residue of three earlier phases of work.

到了第二阶段,Claude 开始设计新增内容。设计过程会经历几次"试错回头",每一次都会在上文里留下痕迹。等到第三阶段,Claude 真正开始写代码时,上下文窗口里已经同时装着"架构探索"、"设计探索"和"新代码写作"——三件事搅在一起。再到"写测试"这一阶段,上下文窗口里占主导的已经变成与测试毫无关系的"历史残渣";而写测试这件事,也就因此受了影响——模型是一边盯着前三个阶段留下的残渣,一边在写测试的。

The Failure Mode Subagents Prevent · subagent 要防止的失败模式

This is the failure mode that subagents exist to prevent. The architecture of a subagent is a direct response to this problem. Each subagent has its own context window. The architectural exploration happens inside an explore subagent, whose context fills with the relevant source code, but only the subagent's final summary returns to the main session. The main session never has to see the dozens of files the subagent read. When the main session moves on to design, it does so with a clean context that contains the summary, not the exploration. When the main session moves on to test writing, the design exploration similarly never enters its context, because the design happened in another subagent. Each phase of work happens in a context that is appropriately scoped to that phase, and the main session preserves a clean overview of the whole project rather than becoming a clearinghouse for every detail of every phase.

这就是 subagent 之所以存在的失败模式。subagent 的架构,正是对这个问题的一种"直球"回应:每个 subagent 都拥有自己的上下文窗口。"架构探索"这件事发生在一个 explore subagent 里——它的上下文会被相关源码填满,但只有它最终的摘要会回到主 session。主 session 根本不必看见那个 subagent 读过的几十个文件。主 session 进入设计阶段时,带着的是一份干净、只含"摘要而非探索过程"的上下文。进入写测试阶段时,"设计阶段的探索"也同样不会进它的上下文,因为设计发生在另一个 subagent 里。每一阶段的工作,都发生在一个恰好匹配该阶段的上下文里;而主 session 始终保持着一份对整个项目的"干净概览",不至于变成"每个阶段每处细节的总收发室"。

Three properties of subagents follow from this architecture, and they are worth naming because every subagent design decision later in this part of the book is a function of one of these three properties.

从这套架构自然引出 subagent 的三条属性。值得给它们取个名字——本部分后面每一个 subagent 的设计决策,都不过是这三条属性中某一条的函数。

The first property is context isolation. The subagent's context is its own. The main session cannot see what the subagent is reading. The subagent cannot see what the main session is doing in parallel. The two communicate only through the subagent's initial brief and the subagent's final result. This isolation is the source of subagents' context-saving power, but it is also a constraint the developer needs to design around. A subagent cannot be told something halfway through its work; whatever it needs to know must be packed into the brief that launches it.

第一条属性是上下文隔离(context isolation)。subagent 的上下文是它自己的。主 session 看不见 subagent 在读什么;subagent 看不见主 session 在并行做什么。两者只通过两件事通信:启动时的 brief 和最终结果。这种隔离既是 subagent 能"省上下文"的力量之源,也是一个开发者必须围绕其做设计的约束——你没法"中途"告诉 subagent 什么新信息;它需要知道的一切,都得在启动它的 brief 里就打包好。

The second property is specialization. A subagent has its own system prompt, distinct from the main session's system prompt. The system prompt establishes the subagent's role, its expertise, its style of thinking, its outputs. A code-reviewer subagent has a system prompt that is tuned for the craft of reviewing code, not for the broader job of being a general programming assistant. A test-writer subagent has a system prompt that is tuned for the craft of writing tests. The narrowness of the system prompt is a feature, not a limitation. A specialist who is asked to do only one kind of work, and who has been briefed in detail about how to do that work well, produces consistently better outputs than a generalist who is asked to do everything.

第二条属性是**专门化(specialization)**。subagent 拥有自己的 system prompt,且和主 session 的 system prompt 截然不同。这份 system prompt 确立 subagent 的角色、专长、思考风格、产出形态。一个 code-reviewer subagent 的 system prompt,是围绕"代码评审这门手艺"调出来的,而不是围绕"当一个通用编程助手"调出来的;一个 test-writer subagent 的 system prompt,是围绕"写测试这门手艺"调出来的。system prompt 的窄,是一种特性,不是限制——被指派只做一类事、并被详细交代过"如何把这事做好"的专员,产出的稳定质量,会高于"什么都做"的通才。

The third property is parallelism. Multiple subagents can run at the same time. Claude Code can run several subagents concurrently; community guidance from Anthropic engineers and from heavy users converges on roughly ten as a practical ceiling before queueing becomes noticeable, although the exact number is environment-dependent and Anthropic has not published a hard cap. This means that work which is genuinely independent can happen in parallel rather than in sequence. A code review that involves a security pass, a performance pass, a style pass, and a test-coverage pass does not have to be four sequential reviews. It can be one parallel review where four subagents work simultaneously and the main session synthesizes their findings when they all return. The wall-clock time savings are substantial. The quality improvements are also substantial, because each subagent is focused on its own dimension and does not have to balance attention across all four.

第三条属性是**并行(parallelism)**。多个 subagent 可以同时跑。Claude Code 能并发跑若干个子代理;来自 Anthropic 工程师和重度用户的社区经验大致收敛在"十个左右"——超过这个数,排队的延迟就开始能被感知;当然具体数字因环境而异,Anthropic 也没有公开过一个硬上限。这意味着:真正独立的工作,完全可以并行发生,而不是顺序发生。一次代码评审,要做"安全"、"性能"、"风格"、"测试覆盖"四道扫描——它不必是依次发生的四次评审,而可以是并行的一次评审:四个 subagent 同时工作,主 session 等它们都返回后再做综合。挂钟时间上的节省相当大;质量上的提升也相当大——因为每个 subagent 都在自己的维度上保持专注,不必把注意力在四个维度间来回分配。

Where Subagents Live · subagent 住在哪里

Where do subagents live, mechanically? They live as files in the `.claude/agents` directory. Project-level subagents live at `.claude/agents` in the root of a specific project, where they apply only inside that project. User-level subagents live at `~/.claude/agents` in the user's home directory, where they apply across every project. The naming convention is the same as for skills: lowercase, hyphenated, descriptive. Each subagent is a single Markdown file with YAML frontmatter at the top and a system prompt body below. Here is the simplest possible subagent definition, complete and ready to install.

subagent 在"机械层面"住在哪里?它们以文件形式住在 `.claude/agents` 目录里。**项目级** subagent 放在某个项目根目录下的 `.claude/agents`,只在那个项目里生效。**用户级** subagent 放在用户家目录的 `~/.claude/agents`,跨所有项目生效。命名约定和 skill 一样:小写、连字符、具描述性。每个 subagent 就是一个 Markdown 文件,顶部是 YAML frontmatter,下面是 system prompt 主体。下面是"最小可用的 subagent 定义",完整、可以直接装上。

Look closely at this file, because every subagent in the rest of this part of the book follows the same shape. The frontmatter has four fields, two of which are required. The `name` field is the subagent's identifier and must match the filename. The `description` field is what tells Claude when this subagent should be dispatched, and it is the subagent equivalent of the skill description; the same craft of trigger writing applies. The `tools` field, optional, restricts the tools available to the subagent. The `model` field, also optional, lets the developer choose which model the subagent runs on, which matters because heavier work might warrant the strongest model while lighter work can use a faster one.

请仔细看这个文件——本部分后面出现的每一个 subagent,都是这个形状。frontmatter 一共四个字段,其中两个是必填。`name` 字段是 subagent 的标识,必须和文件名一致。`description` 字段告诉 Claude"什么时候该派这个 subagent",它相当于 subagent 版的 skill description——trigger 的写法工艺同样适用。`tools` 字段是可选的,用来限制 subagent 可用的工具。`model` 字段也是可选的,让开发者选 subagent 跑在哪个模型上——这一点之所以重要,是因为"重活"值得用最强的模型,而"轻活"可以用更快的那个。

```
---
name: code-reviewer
description: Reviews code for quality and best practices. Use proactively after code changes.
tools: Read, Grep, Glob
model: sonnet
---
You are a senior code reviewer. When invoked, analyze the code you are given and provide specific, actionable feedback.
```

The body of the file is the subagent's system prompt. It is read every time the subagent is dispatched, and it shapes everything the subagent does. The body of a subagent is, in this respect, more consequential than the body of a skill, because while a skill body adds instructions to the main session's overall behavior, a subagent body becomes the entire personality of a separate Claude process. The development of this body is the subject of the next chapter, and it is where the most important design decisions for any subagent get made.

文件的正文,就是 subagent 的 system prompt。每次 subagent 被派出去,这份 system prompt 都会被读入,并塑造它接下来做的所有事。从这个意义上说,subagent 正文的影响,要大于 skill 正文的影响——因为 skill 正文只是给主 session 的整体行为"追加一些指令";而 subagent 正文,会成为另一个 Claude 进程的整体人格。这份正文怎么写,正是下一章的主题——也是任何一个 subagent 最重要的设计决策所在。

Two conventions concerning invocation are worth knowing now. The first is automatic delegation. When the main session is working on a task and notices that one of the available subagents has a description matching the work, the main session can delegate to that subagent on its own initiative. The developer does not need to type any special command to invoke the subagent; she simply describes her task, and the main session decides whether the task should be handled in-line or delegated. This is the same principle that governs skill firing, applied to a different mechanism.

关于"调用",有两条约定值得现在就了解。第一条是**自动委派(automatic delegation)**。主 session 在处理任务时,如果注意到"可用的 subagent 中,某条的 description 匹配当前工作",主 session 就会自主把它派出去。开发者不需要敲任何特殊命令来"召唤" subagent——她只要把任务用自然语言说清楚,主 session 就会自己决定:这件事是就地做,还是委派给某个 subagent。这套原理和"skill 何时触发"是一样的,只是套在了一个不同的机制上。

Good descriptions cause appropriate delegation. Weak descriptions cause subagents to sit unused.

好的 description 会带来恰当的委派;弱的 description 会让 subagent 一直闲坐。

← 前一页

后一页 →

The second convention is explicit invocation. The developer can also invoke a subagent directly, by typing `@agent-name` in the prompt. The mention triggers the named subagent immediately, with whatever brief follows in the prompt as the dispatch instructions. Explicit invocation is useful when the developer wants to be sure a particular subagent handles a particular task, when the description-based delegation might not fire reliably, or when the developer wants to compose multiple specific subagents on a single task. Both invocation modes are valid, and most working developers use both, depending on the situation.

第二条约定是**显式调用(explicit invocation)**。开发者也可以直接在 prompt 里敲 `@agent-name` 来"点名"某个 subagent。这一提,会立刻触发被点名的 subagent,而 prompt 中跟在后面的内容,就是派给它的 brief。显式调用在以下场景特别有用:你希望"某个 subagent 一定要处理某件事"、"靠 description 的委派不够稳"、"你想把多个具体 subagent 组合到一件事上"。两种调用方式都成立——多数实际用起来开发者会两种都用,看场景挑一种。

Automatic Versus Explicit · 自动 vs 显式

The relationship between automatic delegation and explicit invocation is worth a moment of attention, because choosing between them is a real design question. Automatic delegation is convenient. The developer types in plain language and the right specialist is dispatched, much as the right skill is fired. The cost of automatic delegation is reliability. Description-based dispatch is a judgment call that the main session has to make for every prompt, and the call can occasionally be wrong, especially when several subagents have overlapping descriptions. Explicit invocation is reliable. The developer names the subagent she wants and the subagent runs, full stop. The cost of explicit invocation is friction. The developer has to remember the subagent's name and choose to type it. A working pattern that develops in most experienced users is a hybrid: explicit invocation when the work is high-stakes and the developer wants certainty about which specialist runs, automatic delegation when the work is routine and the convenience matters more than the certainty. The hybrid produces the best of both, and it is worth deliberately practicing both invocation modes during the first weeks of subagent use.

"自动委派"和"显式调用"的关系,值得花片刻讲清楚——因为在两者之间做选择,其实是一个真正的设计问题。自动委派很方便:开发者用日常语言敲下去,合适的专员就被派出,和 skill 的"对的 skill 自动触发"差不多。代价是**可靠性**:基于 description 的派单,是主 session 在每个 prompt 上都要做的一次判断,而这个判断偶尔会出错——尤其是当多个 subagent 的 description 互相重叠的时候。显式调用很可靠:你点名哪个 subagent,就是它,全无歧义。代价是**摩擦**:开发者得记得那个 subagent 的名字,并且主动把它敲出来。大多数熟练用户最后会形成一种"混合"模式:高风险工作、要确定是哪位专员跑的,显式调用;常规工作、便利比确定性更重要的,自动委派。这种"混合"兼具两者之利,在开始用 subagent 的头几周里,值得有意识地两种都练一练。

A note on the relationship to skills. Skills and subagents address different problems and complement each other rather than competing. A skill teaches Claude how to do a specific task well. A subagent provides a separate context and a specialized role for a kind of work. Many sophisticated workflows use both: a subagent handles a kind of work, and inside the subagent, skills fire when relevant tasks come up during that work. A test-writer subagent might activate a project-specific test-conventions skill at the moment it begins drafting tests. The two systems

compose, and the composition is where the deepest leverage of Claude Code begins to surface. Chapter Fourteen is dedicated entirely to this composition.

再说和 skill 的关系。Skill 和 subagent 解决的是不同的问题,二者是互补而不是竞争。Skill 教 Claude 如何把一件具体的事做好;subagent 则为某一类工作提供一个独立的上下文和一个专门的角色。许多成熟的工作流会把两者都用上:subagent 负责某一类工作,在它"里头"工作展开时,相关的 skill 又会在该触发的时刻被触发。比如一个 test-writer subagent 在它开始起草测试的那一刻,可能会激活"项目专属测试规约"这个 skill。两套系统是 *可组合的*;而这种组合,才是 Claude Code 最深杠杆的起点。第十四章整章都会讲这种"组合"。

← 前一页

后一页 →

A short clarification for readers who have come from a background of writing prompts and may be confused about why subagents are not simply elaborate prompts. The difference is real and structural. A prompt, no matter how elaborate, runs in the same context as everything else in the session. The context window is shared. The system prompt is shared. A subagent has a separate process, a separate context, and a separate system prompt. The architectural distinction matters because it produces different behavior. A prompt cannot stop the rest of the session from polluting its context. A subagent runs in a clean slate. This is not a difference of degree. It is a difference of kind, and developers who try to replicate subagent benefits with cleverer prompts inside a single session will find that the benefits do not survive the lack of architectural separation. The right answer is to use the architecture, not to try to simulate it.

对那些"来自 prompt 写作背景"的读者,这里要做一个简短的澄清——你可能困惑:为什么 subagent 不就是"复杂版的 prompt"呢?这个区别是真实且结构性的。一个 prompt,无论写得多么复杂,都和 session 里的其他东西跑在同一个上下文里——上下文窗口是共享的,system prompt 是共享的。一个 subagent 则拥有 *独立的进程*、*独立的上下文*、*独立的 system prompt*。这个架构上的差异之所以重要,是因为它会直接造成不同的行为。一个 prompt 挡不住 session 其他部分去"污染"它的上下文;而一个 subagent 跑在干净的白纸上。这不是程度上的差别,而是类型上的差别——想用"更聪明的 prompt"在同一个 session 里模拟 subagent 收益的开发者,会发现那份收益根本撑不住:没有架构上的隔离,收益也就不在了。正解是 *用架构,而不是模拟架构*。

A short word on the relationship to Claude Code's built-in subagents. Three subagents ship with Claude Code by default and are present in every session: Explore, Plan, and General-purpose. Explore is read-only and runs on Haiku by default; it is optimized for quickly searching and understanding a codebase without making changes. Plan gathers context before the main session enters plan mode, where a strategy is presented. General-purpose handles tasks that need a mixture of exploration and modification. The developer does not invoke these directly very often; the main session routes work to them automatically. Their existence is worth knowing because they are a useful proof of concept of the subagent architecture and because, when the developer designs her own custom subagents, she is extending a system that Anthropic itself uses every day.

再点一句和 Claude Code 内置 subagent 的关系。Claude Code 默认自带三个 subagent,每个 session 里都有:Explore、Plan、General-purpose。Explore 是只读的,默认跑在 Haiku 上;它专为"快速搜代码库并理解它"而优化,不会修改任何东西。Plan 则是在主 session 进入 plan mode、准备呈现一份策略之前,先收集上下文。General-purpose 负责"需要既探索又修改"的混合任务。开发者一般不直接点名调它们——主 session 会自动把工作派给它们。值得知道它们的存在,有两个原因:它们是 subagent 架构的一个很好用的"概念验证";而当开发者设计自己的自定义 subagent 时,她是在 *扩展 Anthropic 自己每天都在用的那套系统*。

It is worth examining the cost side of subagents honestly, because the architecture has trade-offs that the developer should understand before building a heavy subagent library. Each subagent invocation costs tokens. The system prompt of the subagent is loaded into a fresh context,

which costs whatever the system prompt's size is. The subagent then reads files, makes tool calls, and produces its final output, all of which consume tokens that the main session would not have consumed if the work had stayed inline. The total token cost of a subagent invocation is, in most cases, somewhat higher than the cost of doing the same work in the main session, because the subagent does not benefit from any context the main session has already built.

值得把 subagent 的成本面也坦诚地摆出来——因为这套架构有取舍,开发者搭一套"重的 subagent 库"之前,理应理解这些。每次调用 subagent 都要花 token:subagent 的 system prompt 会被加载到一份全新的上下文,size 是多少就要花多少;随后 subagent 读文件、调用工具、产出最终输出,这一切都会消耗 token——而这些 token,如果这件事是"在主 session 里直接做"的,本来是不会花的。绝大多数情况下,一次 subagent 调用的总 token 成本,会比"同一份工作在主 session 里做"略高——因为 subagent 享受不到主 session 已经积累起来的任何上下文。

The Cost and Its Compensation · 成本与它的补偿

The compensation for this cost is significant, and it has two components. The first component is the quality improvement that comes from a clean, focused context. A subagent producing a security review in a context that contains only a security-review system prompt and the relevant code will produce a deeper review than a main session that is also juggling architectural decisions, build configuration, and three previous tasks. The quality difference is usually worth more than the token cost, because the developer's time is the scarce resource, not the tokens. The second component is the wall-clock saving from parallelism, which only applies in multi-subagent workflows. When four subagents run simultaneously, the wall-clock time is determined by the slowest of the four, not by the sum. The token cost is roughly four times higher than the equivalent sequential work, but the time cost is one quarter, and time is the dimension that governs how many tasks the developer can complete in a working day.

对这份成本,有两份"补偿",分量都不轻。第一份是来自"干净、专注的上下文"的质量提升。一个 subagent,做安全评审,而它的上下文里只有"安全评审 system prompt + 相关代码",它产出的评审深度,会高于"主 session 同时还忙着架构决策、构建配置、上一二三件事"的那种情形。这种质量差距,通常比 token 成本本身更值——因为真正稀缺的资源是开发者的*时间*,而不是 token。第二份是来自并行的*挂钟时间*节省——它只在多 subagent workflow 里奏效。四个 subagent 同时跑,挂钟时间由其中最慢的一个决定,而不是四者之和。token 成本大约是"等价串行工作"的四倍,但时间成本是四分之一;而"一个工作日里能完成多少任务",正是由时间这个维度决定的。

A balanced view, then, is that subagents are worth dispatching when context isolation matters, when specialization is possible, when parallelism applies, or when several of these are true at once. Subagents are not worth dispatching when the work is small, when the main session's context is already clean, and when no specialization or parallelism is available. The developer who internalizes this balanced view will reach for subagents at the right moments and refrain from over-dispatching them at moments where they would only add overhead.

一个平衡的看法是:在以下场景里,subagent 值得派出——上下文隔离重要、专门化可能、并行适用,或三者同时成立;在以下场景里,subagent 不值得派出——工作很小、主 session 的上下文已经干净、且没有可用的专门化或并行。把这个"平衡看法"真正内化的人,会在对的时刻伸手召唤 subagent,也会在该收手的时刻克制自己——不至于在那些 subagent 只会徒增开销的场合,过度派单。

A second balanced view concerns the human side of subagent use. Some developers, on first encountering subagents, become enthusiastic and try to dispatch them on every task. The result is sometimes slower than the work would have been without them, because the orchestration overhead exceeds the benefits. Other developers, on first encountering subagents, treat them with suspicion and avoid them entirely. The result is

that they never benefit from the architecture and continue to operate inside an increasingly polluted main session. Both extremes are mistakes. The right relationship with subagents, after a few weeks of practice, becomes intuitive: the developer reaches for them when the task has an obvious specialization or parallelism shape, and lets the main session handle the rest. This intuition is not difficult to develop, but it requires a few weeks of deliberate use to settle in. The reader who works through Part Three patiently will arrive at the intuition without having to think about it explicitly any longer.

第二个"平衡看法",是关于人使用 subagent 时的心智层面。开发者第一次接触 subagent 时,有些会变得热情高涨,恨不得每个任务都派——结果有时反而比不派还慢,因为"编排开销"大于收益。也有些开发者第一次接触 subagent 时,满腹狐疑,干脆一个都不用——结果就是他们从来享受不到这套架构的好处,继续在"越来越被污染的主 session"里工作。两个极端都是错的。和 subagent 的正确关系,经过几周实操,会变成一种直觉:遇到任务天然具有"专门化或并行"的形状,就会伸手召唤;其余的事,放给主 session 处理。这种直觉并不难养成,但确实需要几周有意识的练习,才能"安定下来"。耐心走完第三部分的读者,会自然而然地获得这种直觉,不再需要时刻把它挂在嘴边。

The final piece of context for this opening chapter is a property that emerges only after a developer has built and used several subagents, and that is worth naming up front so that the chapters ahead make sense. Subagents are how a single developer becomes the leader of a small specialist team. The team is patient. It does not get tired. It does not have meetings. Its members do not interrupt each other. Each member is excellent at one thing and confidently silent about everything else. The developer's job is to be the editor, the orchestrator, the one who decides who works on what and how the results come together. Claude Code, equipped with a well-designed subagent library, becomes a leverage multiplier in a way that Claude Code without subagents simply cannot. The developer who masters this multiplication will, within a few months of practice, find that the volume and quality of work she can produce has shifted in a noticeable, sustainable way.

本章最后一段上下文,是这样一个属性——它只在开发者搭好、用过若干个 subagent 之后才会浮现,但值得在开篇就先点出来,这样后面几章才好读懂。Subagent,就是"让一个开发者变成一支小型专员团队的 leader"的那条路。这支团队有耐心,不会累,不开会,成员之间不会互相打断;每个成员都只精于一件事,并对其余的事坦荡地"沉默"。开发者的角色,是编辑、是编排者、是那个决定"谁做什么、结果如何汇拢"的人。一套装备了"设计良好的 subagent 库"的 Claude Code,会变成一种**杠杆放大器**——而没装 subagent 的 Claude Code,做不到这种放大。把这种"放大"真正掌握住的开发者,经过几个月的实操之后,会发现自己能产出的工作量和质量,都发生了显著且可持续的迁移。

The chapter that follows takes the body of the subagent file, the system prompt, and treats it as the central design decision in any subagent. Every choice about what a subagent does, what it does not do, how it should think about its work, and how it should report its findings, is a choice expressed in the system prompt. The discipline of writing those prompts well is the discipline that determines whether a subagent is a useful specialist or a confused generalist. That discipline begins now.

下一章会把 subagent 文件的正文——也就是 system prompt——当作任何 subagent 的**核心设计决策**来对待。一个 subagent 做什么、不做什么、它该以怎样的方式思考、它该如何回报它发现的东西——这一切选择,都是写在 system prompt 里的。把这份 system prompt 写好的功夫,就是决定一个 subagent 是"有用的专员"还是"迷茫的通才"的功夫。那门功夫,从下一章开始。

CHAPTER ELEVEN · 第十一章

Designing Effective Subagents: Role, Tools, and Boundaries

设计高效的子代理:角色、工具与边界

"Give a person too much to do, and they do nothing well. Give them one thing, and they may surprise you."

— Peter Drucker, paraphrased

"给一个人太多事,她什么也做不好。只给她一件事,她可能给你惊喜。"

——彼得·德鲁克(转述)

A subagent that does too much is worse than no subagent at all. This sounds counterintuitive on first hearing, because the natural instinct of a developer encountering a new capability is to put as much capability into it as possible. The instinct is wrong, and this chapter explains why and what to do instead. The art of subagent design is the art of restraint. The subagent that performs one role well, with a tightly defined scope and a thoughtfully constrained set of tools, produces consistent, reliable, useful output. The subagent that tries to be a general assistant in a slightly different costume produces output that is sometimes excellent, sometimes mediocre, and almost never reliably one or the other. Reliability is the entire point.

一个"做太多"的 subagent,比"没有 subagent"更糟。这话初听像是反直觉——因为开发者面对一项新能力时,本能反应就是把尽可能多的能力塞进去。这个本能是错的。本章会解释它为什么错,以及正确做法是什么。Subagent 设计的艺术,本质是克制的艺术。一个把单一角色做好、范围严格、工具精心收窄的 subagent,产出是稳定、可靠、可用;一个"换了一身衣服的通用助手"型 subagent,产出偶尔精彩、偶尔平庸,而且几乎从来不会稳定地只属于其中一种。可靠性,就是全部意义所在。

Three design dimensions govern every subagent, and the disciplined consideration of each is what separates good subagents from poor ones. The first dimension is role. The second is tools. The third is boundary. Each of these expresses a constraint, and constraints are what make subagents useful. A subagent without constraints is, in effect, a slightly more isolated version of the main session, and most of the value of having a subagent is forfeited.

三个设计维度决定每一个 subagent 的好坏——对每一个维度"有纪律地思考",正是好与差的分水岭。第一个维度是**角色(role)**,第二个是**工具(tools)**,第三个是**边界(boundary)**。三者各表达一种约束——而约束,正是 subagent 之所以有用的原因。一个没有约束的 subagent,说穿了只是"主 session 的稍隔离版本";拥有 subagent 的大半价值,会在这种设计里被浪费掉。

Begin with role. The role of a subagent is its identity, expressed at the top of its system prompt. A code-reviewer subagent is a code reviewer. Not a general programmer who reviews code sometimes. Not a teacher who happens to be reviewing code. A code reviewer. The role definition should be specific enough that it constrains the kind of work the subagent does, the kind of language it uses, the kind of judgment it brings, and the kind of output it produces. A weak role definition reads like a job title. A strong role definition reads like the introduction of a person.

从**角色**说起。subagent 的角色,就是它的"身份",写在 system prompt 的最前面。一个 code-reviewer subagent,就是一位代码评审者——而不是一位"偶尔评审代码的通用程序员",也不是一位"恰好在做评审的老师"。它就是**代码评审者**。角色定义要足够具体,具体到:它能约束这个 subagent 做什么类型的工作、用什么类型的语言、做哪种判断、产出什么形态的输出。弱的角色定义,读起来像一行岗位名;强的角色定义,读起来像一个人的开场介绍。

Compare these two openings of a code-reviewer subagent system prompt. The first is weak.

对比下面两段 code-reviewer subagent 的 system prompt 开场。第一段是弱的。

You are a code reviewer. Review code carefully.

Strong Subagent Descriptions · 强的 subagent 描述

The second is strong.

第二段是强的。

You are a senior code reviewer with ten years of experience reviewing pull requests on production systems. Your job

The second prompt does many things at once. It establishes seniority. It establishes purpose. It explicitly rejects two common failure modes: empty friendliness and padded reviews. It gives the subagent a posture, a stance toward its work. The subagent that runs under the second prompt produces noticeably different output from the subagent that runs under the first, even though both are nominally code reviewers. The role definition is a substantial fraction of the subagent's personality, and writing it carefully is among the most consequential acts of subagent design.

第二段 prompt 一次做了很多事:确立资历、确立目的、显式拒绝两种常见失败模式(空洞的客套和注水式评审)、并给 subagent 立了一种姿态——一种对待自己工作的站位。跑在第二段 prompt 之下的 subagent,产出的输出和跑在第一段之下的明显不同——尽管两者名义上都是"代码评审者"。角色定义,占了 subagent 人格的相当大一块;把它写好,是 subagent 设计中分量最重的工作之一。

Move to tools. The tools field in the frontmatter restricts which tools the subagent is allowed to invoke. Omitting the field grants the subagent the full set of tools available to the main session. Specifying a list narrows the set. The narrowing is not a security feature alone, although it is a useful one. The narrowing is also a focusing feature. A subagent that has only the tools it needs does not get distracted by the tools it does not need. A code-reviewer subagent that has access only to Read, Grep, and Glob cannot make file edits even if its system prompt forgets to forbid them, and the absence of Edit, Write, and Bash from its toolkit shapes how it approaches every task. The subagent does not casually offer to fix the issues it finds. It cannot, because the tools to do so are not present, and so its output naturally reflects the boundary.

接下来说工具。frontmatter 里的 `tools` 字段,用来限制 subagent 可以调用哪些工具。不填这个字段,subagent 就拿到主 session 的全套工具;填上列表,工具就被收窄。收窄,不仅仅是一个安全特性(虽然它确实有用)——它更是一个聚焦特性。一个只拥有"它真正需要的工具"的 subagent,不会被"它不需要的工具"分心。一个只拥有 Read、Grep、Glob 的 code-reviewer subagent,即使 system prompt 忘了写"不许改文件",它也改不了——因为 Edit、Write、Bash 根本不在它的工具箱里,这份"不在"塑造了它对待每一项任务的方式。它不会随口提议"顺手把它发现的问题修了",做不到,工具不在,于是它的输出自然就反映了这条边界。

A simple rule guides tools restriction. Begin with no tools, then add only the ones the subagent demonstrably needs. A reviewer needs to read files, search them, and find them, so Read, Grep, and Glob suffice. A documentation-writer needs to read files and write new ones, so Read, Write, and Glob suffice. A test-writer needs to read existing code, write new test files, and possibly run tests to verify, so Read, Write, Bash, and Glob form a reasonable set. The principle in each case is that the tool list mirrors the subagent's job description. If the subagent does not need a tool to do its job, the tool is not granted, full stop.

工具限制的规则很简单:从"零工具"开始,只加那些"被证明确实需要"的工具。评审者要"读、搜、找文件",所以 Read、Grep、Glob 就够了。文档作者要"读旧文件、写新文件",所以 Read、Write、Glob 够用。测试作者要"读已有代码、写新测试文件、可能要跑测试验证",所以 Read、Write、Bash、Glob 算合理组合。每一种情形,道理都一样:工具列表要"镜像" subagent 的岗位描述。它完成工作不需要的工具,一律不授予,完。

← 前一页

后一页 →

Move to boundary. The boundary of a subagent is the set of things it explicitly does not do. The role definition says what the subagent is. The boundary says what the subagent is not. Both are necessary, because the language model that runs the subagent will, when uncertain, tend to drift toward generality, and explicit boundaries push back against that drift.

接下来说**边界**。subagent 的边界,是它"显式不做事"的集合。角色定义说的是"这个 subagent 是什么";边界说的是"它不是什么"。两者都必要,因为跑这个 subagent 的语言模型,在不确定时,会倾向于向"通用"漂移——而显式边界,正是把这种漂移顶回去的力量。

Boundary Language in Prompts · prompt 里的边界语言

Boundary language belongs in the system prompt and should be specific. A code-reviewer subagent should have boundary language such as: you do not write or modify code; if you find an issue, you describe it and recommend a fix, but you do not implement the fix yourself. A test-writer subagent should have boundary language such as: you do not change the code under test; if you discover a bug while writing tests, you note it in the test as a comment, and you flag it in your final summary, but you do not modify the production code. A documentation-writer subagent should have boundary language such as: you do not modify code, configuration, or any non-documentation file. Each of these statements feels redundant when first written, because the developer thinks the role definition already implies the boundary. The redundancy is the point. Stating the boundary explicitly costs nothing and prevents the small but real failure mode of a subagent quietly drifting outside its scope on a task where the role definition was ambiguous.

边界语言属于 system prompt,而且要写得具体。code-reviewer subagent 应该有这样的边界语言:"你不写也不改代码;如果你发现了问题,把它描述出来并建议怎么改,但不要自己去改。"test-writer subagent 应该有这样的边界语言:"你不改被测代码;如果你在写测试的过程中发现了一个 bug,在测试里以注释记下来,并在最终总结里 flag 出来,但不要去改生产代码。"documentation-writer subagent 应该有这样的边界语言:"你不修改代码、配置、任何非文档文件。"每一条写下来时,第一感觉都是"啰嗦"——因为开发者觉得,角色定义已经把这件事隐含说清了。这份"啰嗦"恰恰是重点:显式写出边界,几乎没有成本,却能避免一种小而真实的失败模式——在角色定义本就模糊的任务上,subagent 悄悄滑出自己的职责范围。

A complete subagent file, designed under all three dimensions, looks like this. The example below is a security-auditor, intended to focus narrowly on security issues in code, leaving general code review and other concerns to other subagents.

一个完整的 subagent 文件,按这三个维度设计出来,长这样。下面的例子是一个 security-auditor,设计目标是把"代码里的安全问题"这件事做窄,把通用代码评审和其他关切留给其他 subagent。

```
---
name: security-auditor
description: Audits code for security issues, including OWASP Top 10 vulnerabilities, hardcoded secrets, unsafe dependencies
tools: Read, Grep, Glob, Bash
model: opus
---
You are a senior application security engineer. Your job is to find security issues in code that other reviewers are
```

← 前一页

后一页 →

Read the file once more, slowly, and notice the discipline at every level. The description in the frontmatter is specific about the kind of work the subagent does and uses the *proactively* phrasing recommended by Anthropic to encourage reliable delegation. The tools list is narrow, exactly what the subagent needs and nothing more. The model is set to opus, because security review benefits from the strongest model the developer has available. The system prompt establishes a senior role with explicit areas of expertise. It defines the method as a numbered procedure. It includes a what-you-do-not-do section that is as long as the what-you-do section, because the boundary is genuinely as important as the role. It ends with a directive about the format of every report. Every degree of freedom that could produce variability in output has been pinned down deliberately.

请把这份文件再慢慢读一遍——留意每一层上的纪律。frontmatter 里的 description,具体到它要做的那类工作,并用了 Anthropic 推荐的 *proactively* 措辞,以促成稳定可靠的委派。工具列表很窄,正是它需要的那些,多一项没有。模型设成 opus,因为安全评审值得用开发者手边"最强的那一档"。system prompt 确立一个高级角色,并明确列出"专长领域";它把"方法"定义为一套带编号的流程;它包含一段"你不做什么",篇幅和"你做什么"相当,因为边界和角色一样重要;它以一段"每份报告该长成什么样"的指令收尾。每一种可能带来输出差异的自由度,都被有意识地钉死了。

```
Your areas of expertise:
- Authentication and authorization flaws
- Injection vulnerabilities (SQL, command, prompt, template)
- Hardcoded secrets and credentials
- Unsafe deserialization
- Missing input validation
- Insecure direct object references
- Cross-site scripting and request forgery in web contexts
- Time-of-check to time-of-use bugs
- Vulnerable or outdated dependencies

Your method:
1. Read the code you are given.
2. Look for the patterns above explicitly. Do not skim.
3. For each issue you find, produce a short report containing:
- Severity: critical, high, medium, or low
- Location: file path and line number
- Description: what the issue is, in plain language
- Recommendation: a concrete fix, with a code example if useful

What you do not do:
```

- You do not modify code. You only report.
- You do not perform general code review. Style, performance, and architecture concerns belong to other reviewers.
- You do not pad your output. If the code has no security issues, say so plainly in one sentence.
- You do not invent issues. If you are uncertain whether something is a vulnerability, mark it as a question rather

End every report with a one-sentence summary judgment: clear, minor issues only, or significant issues found.

← 前一页

后一页 →

A subagent designed this carefully produces strikingly consistent output across many invocations. Two different developers running this subagent against two different codebases will get reports that look like they came from the same security engineer, because in a meaningful sense they did. This consistency is what makes subagents useful as components of a larger system. A subagent whose output varies wildly from invocation to invocation is hard to compose with other subagents, hard to review, and hard to trust. A subagent whose output is stable in form, even when the substance varies with the codebase, can be plugged into a larger workflow with confidence.

这样用心设计出来的 subagent,在多次调用间会产出惊人地一致的输出。两位不同的开发者,把同一个 subagent 跑在两个不同的代码库上,会拿到"看起来像出自同一位安全工程师"的报告——因为在有意义的层面上,本来就是同一位。这种一致性,正是 subagent 之所以能成为"更大系统的组件"的原因。输出每次大变的 subagent,很难和其他 subagent 组合、很难复盘、也很难被信任;输出形态稳定(只是内容随代码库而变)的 subagent,可以放心地嵌入到更大的工作流里。

A second worked example will reinforce the design principles, because the discipline becomes habitual only after the developer has seen it applied to several different specialties. The subagent below is called `documentation-writer`. Its job is to produce or update documentation for code, with the same kind of consistency and discipline as the security-auditor.

第二个完整例子,用来再强化一遍设计原则——这种纪律,要等开发者看过它在不同专长上反复套用,才会真正变成习惯。下面这个 subagent 叫 `documentation-writer`。它的职责是产出或更新代码的文档,和 `security-auditor` 一样,追求一致性和纪律性。

```
---
name: documentation-writer
description: Produces clear, useful technical documentation for code, modules, or APIs, including README sections, J
tools: Read, Write, Glob, Grep
model: sonnet
---
You are a senior technical writer who works alongside engineers. Your job is to produce documentation that engineers
```

← 前一页

后一页 →

Compared with the security-auditor, the documentation-writer differs in several telling ways. The role is different: technical writer rather than security engineer. The tools are different: Read, Write, Glob, Grep, including the ability to write files because documentation is the deliverable; security-auditor was read-only because audit is its deliverable. The model is sonnet rather than opus, because documentation work does not particularly benefit from the strongest model and the cost saving is real for a subagent that may run several times a day. The principles section is where the writer's craft is encoded; the security-auditor had a similar section, but its content was about security, not about writing.

和 security-auditor 相比,documentation-writer 在几个有意义的点上不同:角色不同——技术写作者,不是安全工程师;工具不同——Read、Write、Glob、Grep,这里需要"能写文件",因为文档本身就是交付物;而 security-auditor 是只读的,因为"审计报告"才是它的交付物。模型是 sonnet 而非 opus——因为文档写作并不特别依赖"最强档",而一个可能每天跑好几次的 subagent,省下来的成本是真金白银。"原则"那一节,是把"写作者的手艺"编码进去;security-auditor 也有类似的一节,但内容是关于"安全"的,不是关于"写作"的。

Your principles:

- Documentation explains why, not what. Anyone can read the code to see what it does. Documentation should explain why.
- The audience is a smart engineer who has not seen this code before and who will spend roughly five minutes deciding if it is worth reading.
- Examples beat prose. A short worked example is worth several paragraphs of explanation.
- Honest documentation beats aspirational documentation. Document what the code actually does today, not what you wish it did.
- Concise documentation is read. Long documentation is skimmed at best and ignored at worst.

Your method:

1. Read the code in full. Read the surrounding code if needed for context.
2. Identify the public surface of the code: what callers will use.
3. Produce documentation in the project's existing format. Inspect the project to determine what format is in use (Read the code, not the documentation).
4. Cover, in order: purpose, public API, expected inputs and outputs, important behaviors and edge cases, a short worked example.

What you do not do:

- You do not modify code. Your edits are confined to documentation files.
- You do not write documentation that is a transliteration of the code into English.
- You do not invent behaviors that the code does not exhibit.
- You do not pad. A short, accurate page is the goal.
- You do not produce sycophantic prose. Avoid phrases like "powerful," "elegant," or "robust" unless the developer wrote them.

End with a list of the documentation files you wrote or updated, plus a short note on anything you found unclear in the code.

← 前一页

后一页 →

What the two subagents share is the structural discipline. Both have a clear role definition. Both have a numbered method. Both have an explicit list of things they do not do. Both have a directive about the format of their final output. This shared structure is the design discipline of the chapter, and the reader should recognize that the structure itself is doing the work of producing reliability, even more than any specific instruction the prompts contain.

这两位 subagent 共享的,是结构上的纪律:都有清晰的角色定义,都有带编号的方法,都有一份"明确列出的不许做事项",都有关于"最终输出格式"的指令。这种共享的结构,正是本章的设计纪律;读者应当意识到,真正在产出"可靠性"的,正是结构本身——比 prompt 里任何具体指令都更甚。

A debugging walkthrough is worth including, because the experience of building subagents that misbehave is universal, and the cure is teachable. The walkthrough below is from a real session. The developer had built a code-reviewer subagent, dispatched it on a pull request, and

found that the output was not what she expected. The subagent had produced a review, but the review felt generic. It mentioned reasonable things, but those reasonable things did not seem to come from the specific code the subagent had been asked to review. The developer suspected, correctly, that the subagent had read the code in a cursory way and then produced a review that could have applied to almost any code.

"调试式走查"值得一节——"搭一个 subagent 却跑出来不对"这种经历,是普遍的;而它的解法是可教的。下面这段来自一次真实的 session:开发者搭好一个 code-reviewer subagent,把它派去评审一个 PR,发现输出和预期不符。subagent 给出了评审,但那份评审"很通用"——它提到了不少合理的事,但这些事看起来不像是"针对被评审的那份具体代码"出来的。开发者怀疑,而且怀疑对了——subagent 只是蜻蜓点水地读了读代码,然后产出了一份"放在哪段代码上都说得通"的评审。

She investigated the system prompt. The role definition was decent. The method was a numbered procedure. The output format was specified. But on close reading, she noticed that the method said "review the code" without instructing the subagent to read it carefully, look for specific issues, or reference specific lines. The boundary section did not say what generic feedback was forbidden. The result was that the subagent, working under permissive instructions, took the easiest path and produced output that was technically a review but not the close, specific review the developer wanted.

她去检查 system prompt。角色定义还行,方法是带编号的流程,输出格式也指定了。但她细看之下发现:"方法"那一节只说"review the code",并没有指示 subagent 仔细读、去找具体问题、引用具体行;边界那一节也没说"不许给通用反馈"。结果就是:subagent 拿着相当宽松的指令,挑了最容易走的路,产出的输出技术上确实是"一份评审",但不是开发者想要的那种"贴着代码、具体到位"的评审。

She rewrote the method. The new method instructed the subagent to read the diff in full before forming any opinions, to identify three to five specific issues that referenced file paths and line numbers, and to write each comment as it would appear in a real pull request thread. She added a what-you-do-not-do entry forbidding generic observations that did not reference specific lines. She redispached the subagent on the same pull request. The output transformed. The new review named four specific issues, each with a file and line reference, each with a concrete suggestion. The genericity was gone. The total time spent on the fix was about twenty minutes, including the time to test the new version on three different inputs to confirm the consistency had improved.

她重写了"方法"。新版方法要求 subagent 在形成任何意见之前,先把 diff 完整读完;要求它找出三到五条具体问题,且每条都要带文件路径和行号;还要求它把每条意见写成"在真实 PR 评论流里"那种样子。她又在"你不做什么"那一节加了一条,禁止出现"不引用具体行"的通用观察。然后她把 subagent 重新派去同一个 PR。输出脱胎换骨。新评审点出了四个具体问题,每个都带文件与行号,每个都带具体建议。"通用感"消失了。整次修复花的时间大约二十分钟,其中还包括在三个不同输入上跑新版本来确认一致性已经改善。

← 前一页

后一页 →

A Debugging Lesson · 一条调试心得

The lesson the developer took away from this debugging session was that subagent quality is largely controlled by the specificity of the instructions in the system prompt. A subagent given vague instructions produces vague work. A subagent given specific, structured instructions, with explicit prohibitions on the failure modes the developer has already encountered, produces work that consistently lives up to the design. The fix is rarely to make the subagent smarter. The fix is almost always to make the instructions more specific.

这位开发者从这次调试里带走的心得是:subagent 的质量,几乎完全由"system prompt 里指令的具体程度"决定。给一份含糊的指令,得到的是含糊的产出;给一份具体、有结构、且对"已经踩过的失败模式"有显式禁止的指令,产出就能稳定地配得上设计者的预期。修法的方向很少是"让 subagent 更聪明",几乎总是"让指令更具体"。

A final practice deserves mention before this chapter ends, because it is the practice that separates careful subagent design from sloppy subagent design. After writing a subagent, the developer should test it on three different inputs and compare the outputs. Not for correctness, although correctness matters. For consistency. The three outputs should look like they came from the same specialist. Same format. Same tone. Same level of detail. If the outputs vary in any of these dimensions, the system prompt has not pinned down the relevant degree of freedom, and the system prompt should be tightened until the variation disappears. This three-input test takes ten minutes and prevents an entire category of long-tail problems that would otherwise only surface in a real session at an inconvenient moment.

本章收尾前,还有一项实践值得提——它是"用心的 subagent 设计"和"潦草的 subagent 设计"的分水岭。写完一个 subagent 之后,开发者应该在三个不同的输入上跑一遍,对比输出。不是为了看正不正确(虽然正确也重要)——而是为了看一致不一致。三份输出看起来应该像出自同一位专员:同样的格式、同样的语气、同样的细节水平。哪个维度上出现了摇摆,system prompt 就还没把那个自由度钉死,就要把 prompt 收紧,直到那份摇摆消失。这个"三输入测试"只要十分钟,却能预防一整类"长尾问题"——那些问题,本来只会某个不方便 session 时刻才真正浮出水面。

Subagent design, like skill design, becomes faster with practice. The first subagent the developer writes will take an hour. The fifth will take twenty minutes. The tenth will take ten minutes, because the design dimensions and the system-prompt patterns are now familiar, and the developer has a working library of prior subagents she can adapt. The path from beginner to confident designer is short, and the rewards begin compounding from the first well-designed specialist.

Subagent 设计,和 skill 设计一样,熟能生巧。开发者写的第一个 subagent 大概花一小时,第五个只要二十分钟,第十个只要十分钟——因为设计维度和 system prompt 模板已经熟,而且手边已经有一套可改造复用的过往 subagent 库。从新手到"自信的设计者"这条路不长;而回报,从第一个被精心设计的专员开始,就在复利了。

A short reflection on the role of the model field in subagent design. The model field in the frontmatter lets the developer specify which model the subagent runs on. This may seem like a minor detail, but in practice it is one of the more consequential design choices. Subagents that demand the strongest reasoning, such as the security-auditor and the architecture-reviewer earlier in this part of the book, benefit from running on opus. Subagents that perform structured but less reasoning-intensive work, such as the documentation-writer or the test-writer, run perfectly well on sonnet at lower cost. Subagents that perform fast pattern-matching work, such as a search-helper or an explore agent, can run on haiku and complete in seconds rather than minutes. The model field, used thoughtfully, lets the developer tune each subagent to the cost and speed profile that best fits its job, rather than running everything on a single model and either overspending on simple work or underspending on demanding work. This kind of per-subagent tuning is the small mark of a mature subagent library, and it pays for itself across the months a library is in use.

简短说一说 `model` 字段在 subagent 设计里的角色。frontmatter 里的 `model` 字段,让开发者指定 subagent 跑在哪一档模型上。这看起来像是细节,但在实操中,它是分量更重的设计选择之一。需要最强推理的 subagent——比如前面提过的 security-auditor 和 architecture-reviewer——跑在 opus 上是值得的。做"结构化、但推理强度不高"的工作的 subagent——比如 documentation-writer、test-writer——完全可以在 sonnet 上跑,成本更低。做"快速模式匹配"的 subagent——比如一个搜索助手或 explore agent——可以跑在 haiku 上,几秒而不是几分钟搞定。带着这份心思去用 `model` 字段,就能给每一个 subagent 调出"最适合它工作的成本-速度组合"——而不是把一切压在一档模型上,要么在简单工作上多花,要么在吃力的工作上少花。这种"逐 subagent 调档",是一套"成熟 subagent 库"的小印记;而它在今后数月的使用里会自己把本钱赚回来。

The next chapter takes subagent design from the single subagent to the multi-subagent workflow. With the design of an individual specialist established, the question becomes how multiple specialists work together on a single task, how the main session orchestrates them, and how parallel execution turns work that would have been sequential into work that finishes in a fraction of the time. Orchestration is where the architecture of subagents reveals its full power, and that power is the subject ahead.

下一章会把 subagent 设计,从"单个 subagent"推进到"多 subagent 工作流"。当"单个专员的个体设计"已经确立,问题就变成:多个专员如何在一项任务上协作?主 session 如何编排它们?并行执行如何把"本来是顺序"的工作,变成"挂钟时间只是其中一小段"的工作?编排(orchestration),正是 subagent 架构展现其全部力量的地方——这份力量,正是下一章的主题。

← 前一页

后一页 →

CHAPTER TWELVE · 第十二章

Parallel Subagents and the Orchestrator Pattern

并行子代理与编排者模式

"Coordination is the art of getting many things done in the time of one. The conductor does not play any of the instruments."

— Folk wisdom of the orchestra pit

"协调的艺术,就是让许多件事在"一件事的时间里"做完。指挥家自己不演奏任何一种乐器。"

—— 乐池里流传的民间智慧

The previous chapter taught the design of a single subagent. This chapter is about what happens when multiple subagents work on the same task at the same time. Parallelism is the property that turns subagents from a useful refinement into a structural change in how a developer relates to her tool. A single subagent saves context. Multiple parallel subagents save wall-clock time, deepen quality through specialization, and produce reviews and analyses that no single Claude session, no matter how capable, could produce in the same span of minutes.

上一章讲的是"单个 subagent"的设计。本章讲的是"多个 subagent 同时做一件事"时会发生什么。并行(parallelism)这条属性,把 subagent 从"一项有用的精修"变成"开发者与她工具之间关系的结构性改变"。一个 subagent 节省上下文;多个并行 subagent 节省挂钟时间、通过专门化深化质量、并产出那种"任何单个 Claude session、再怎么强,也不可能在同一段分钟数里做出"的评审与分析。

Begin with the simple case. The main session is asked to perform a task that has multiple genuinely independent components. The components do not need to share information with each other, and the order in which they are performed does not affect the result. A code review of a substantial pull request is a perfect example. A reviewer worth her salt will check several dimensions: correctness, security, performance, style, test coverage. None of these dimensions strictly depends on any other. A correctness check does not need the security check to have run first. A

style check does not need the performance check to be complete. The dimensions are orthogonal, which is exactly the property that makes them candidates for parallel execution.

先看最简单的那种情形:主 session 接到一项任务,任务包含若干个真正独立的组件。这些组件之间不需要共享信息,执行顺序也不影响结果。一次对大型 PR 的代码评审,就是个完美例子。一名有分量的评审者会从几个维度去查:正确性、安全、性能、风格、测试覆盖。这些维度之间没有严格的依赖关系——正确性检查不需要等安全检查先做完,风格检查也不需要等性能检查先完成。各维度是正交的——这种正交性,恰恰是它们适合被并行执行的原因。

The orchestrator pattern formalizes the idea. The main session takes the role of conductor. It receives the task, decomposes it into independent components, dispatches one subagent per component, waits for them all to finish, and synthesizes their outputs into a unified result. The subagents do not speak to each other. They do not know about each other. Each one focuses on its single dimension, produces its single bounded report, and returns. The conductor takes all the reports, weaves them together according to whatever format the unified output requires, and presents the result to the developer. The developer sees one coherent answer. The work that produced the answer happened in parallel, in five separate contexts, each tuned to its own dimension.

编排者模式(orchestrator pattern)把这个想法形式化。主 session 担任指挥家角色:它接下任务,把它拆成若干个独立组件,为每个组件派一个 subagent,等它们全部跑完,再把这些输出综合成一份统一结果。subagent 之间互不说话、互不知道彼此。每个 subagent 都聚焦于自己的那一维,产出自己那份边界清晰的报告,然后返回。指挥家把所有报告收齐,按"统一输出所需的格式"编织成一份,再交给开发者。开发者看到的是一份连贯的答案;而这份答案背后发生的工作,是并行发生的——发生在五个相互独立、各自针对自己维度的上下文里。

← 前一页

后一页 →

A worked example will make this concrete. The developer has a pull request she wants reviewed thoroughly before merging. She types into Claude Code something like, give me a full review of this PR, including security, performance, style, and test coverage. The main session recognizes that a thorough review is being requested, recognizes the dimensions involved, and dispatches four subagents in parallel: a security-auditor, a performance-reviewer, a style-reviewer, and a test-coverage-checker. Each subagent receives the same brief: review this diff, focused on your dimension. Each subagent runs in its own context, reads the relevant files, applies its specialty, and produces its bounded report. The four reports return to the main session. The main session presents them in a unified format, with a summary judgment that integrates the findings across the four dimensions.

一个完整例子会让这件事更具体。开发者有一个 PR,想在合并前被彻底评审一遍。她在 Claude Code 里敲一句类似的话:"给我一个完整的 PR 评审,覆盖安全、性能、风格、测试覆盖。"主 session 识别出"用户要的是一次彻底评审",识别出里面涉及的维度,于是并行派出四个 subagent:security-auditor、performance-reviewer、style-reviewer、test-coverage-checker。每一个 subagent 收到的 brief 都一样:评审这份 diff,聚焦于你负责的维度。每个 subagent 跑在各自的上下文里,读相关文件,发挥自己专长,产出边界清晰的报告。四份报告回到主 session;主 session 用统一格式呈现它们,并给出一段"把四维发现整合在一起"的总结判断。

Estimating the Wall-Clock Saving · 估算挂钟时间的节省

The wall-clock saving is straightforward to estimate. If each subagent takes two minutes, four subagents in sequence take eight minutes, while four subagents in parallel take roughly two minutes, plus the synthesis step. The factor of four speedup is real and felt. On a code review that the

developer might run twice a day, the saving across a working week is substantial.

挂钟时间上的节省,直白到可以用算术算出来:每个 subagent 跑两分钟,顺序跑四个要八分钟,并行跑四个加上综合那一步大约两分钟。这 4 倍的加速是实在的、感受得到的。评审这件事一天可能跑两次,折算到一个工作周,节省下来的时间是可观的。

But the time saving is not the most interesting benefit. The more interesting benefit is the quality. Each subagent operates in a context that contains only the material relevant to its dimension. The security-auditor's context contains the diff plus the security-auditor's system prompt, focused on security. The performance-reviewer's context contains the diff plus the performance-reviewer's system prompt, focused on performance. Neither subagent has its attention diluted by considerations that are not its own. The result is that each subagent produces a deeper, more focused report than a single generalist reviewer could produce on the same input. The total review is not just faster; it is better, because each part of it was done by a specialist who was not trying to balance attention across four dimensions at once.

但更值得讲的不是时间,而是*质量*。每个 subagent 工作的上下文里,只有"与它这一维相关"的材料。security-auditor 的上下文里只有 diff 加它自己的 system prompt,聚焦在安全上;performance-reviewer 的上下文里也只有 diff 加它自己的 system prompt,聚焦在性能上。没有任何一个 subagent 的注意力被"不属于自己的考量"稀释。结果就是:每个 subagent 产出的报告,都比"一个通才评审者面对同一份输入"所产出的更深、更聚焦。整个评审不仅更快,还更好——因为它的每一段都出自一位"不用同时兼顾四个维度"的专员之手。

The orchestrator pattern has a discipline that is worth naming. The main session's job, as the orchestrator, is decomposition and synthesis. The subagents' job is the specialty work. The boundary between these two jobs should be respected. The main session should not attempt to do specialty work itself, even when it could; if a security review is needed, the security-auditor subagent should do it, not the main session. The main session should not attempt to mix specialties inside a single subagent, either; a security-and-performance reviewer is a worse subagent than two narrow ones, because the specialization advantage is forfeited. The orchestrator pattern asks the developer to design subagents narrowly, then trust the design, and let parallelism do its work.

编排者模式里有"纪律"二字值得点出。主 session 作为编排者,做的事是*分解与综合*;subagent 做的事是*专长工作*。这两件事之间的边界要守住。主 session 不该"哪怕能做也亲自去"做专长工作——需要安全评审,就派 security-auditor,而不是主 session 自己上。主 session 也不该在某个 subagent 内部"混专长"——一个"既安全又性能"的评审 subagent,比两个窄专长的 subagent 更差,因为"专业化"带来的优势被白白扔掉。编排者模式要求开发者把 subagent 设计得窄、然后信任这份设计,让并行去做它该做的事。

← 前一页

后一页 →

There are limits to parallelism worth knowing. Claude Code can run several subagents concurrently in practice; community guidance from heavy users converges on roughly ten as a practical ceiling before queueing becomes noticeable, although Anthropic has not published a hard cap. Beyond that limit, additional dispatches queue and run as earlier ones finish. Within the limit, the parallelism is genuinely simultaneous: the subagents do not take turns; they execute in independent processes. The cost of running multiple subagents is, naturally, multiplied by the number of subagents. A four-subagent review costs roughly four times the tokens of a single-subagent review. This cost is usually well worth paying, because the time and quality benefits dominate, but it is a real cost and worth being aware of.

并行的边界值得知道。实践中,Claude Code 能并发跑若干个 subagent;重度用户的社区经验大致收敛在"十个左右"——超过这个数,排队延迟就开始能被感知;不过 Anthropic 并没有公开过一个硬上限。超过这个上限,新的派单会进入队列,等前面跑完再出。在上限以内,并行是真正"同时"的:subagent 之间不轮换,跑在彼此独立的进程里。同时跑多个 subagent 的成本,自然也按数量倍增。一次四

subagent 评审的 token,大约是一次单 subagent 评审的四倍。这份成本大多数情况下完全值得付,因为时间与质量上的好处压过它;但它终究是实在的成本,值得心里有数。

There are tasks for which parallelism is the wrong shape, and recognizing them is part of the discipline. Tasks whose components depend on each other do not benefit from parallel execution; the dependencies force sequential work. A task that involves first reading a database schema, then writing a query against it, then optimizing the query, has a strict order that cannot be parallelized usefully. Tasks whose components share a great deal of context also do not benefit; if every subagent would need to read the same fifty files to do its job, the duplication of that reading across many subagents wastes tokens and time. The right pattern in the dependent case is sequential subagent dispatch, where the output of one subagent becomes the brief for the next. The right pattern in the shared-context case is sometimes a single subagent that handles all the work, or sometimes a careful brief that summarizes the shared context once and passes the summary to each subagent so they do not all need to re-derive it.

有些任务天生不适合并行,识别出它们也是纪律的一部分。组件之间有依赖的任务,无法从并行里获益——依赖关系把工作强行串成顺序。一件"先读数据库 schema、再写查询、再优化查询"的任务,就有严格的顺序,无法有意义地并行。组件之间共享大量上下文的任务,也无法从中获益——如果每个 subagent 都要先读同一份 50 个文件,这种"每个 subagent 都重读一遍"对 token 和时间都是浪费。对"有依赖"的情况,正确模式是顺序派单:上一个 subagent 的输出,成为下一个 subagent 的 brief。对"共享上下文"的情况,正确模式有时是派一个 subagent 把活全干了;有时是写一份精心准备的"共享上下文摘要",一次性传给每个 subagent,让它们不必各自重新推导。

A useful framing for deciding when parallelism applies is this question: would I, as a developer, hire four different specialists for four different parts of this task, knowing that the specialists do not need to talk to each other? If the answer is yes, the orchestrator pattern fits. If the answer is no, because the specialists would need to coordinate continuously, the work belongs in a single subagent or in a sequence. This human-team metaphor maps cleanly onto subagent design, and developers who train themselves to ask the metaphor question quickly learn to recognize which workflows benefit most from parallelism.

判断"什么时候适合并行"有一个好用的框架——问自己这样一个问题:"我作为开发者,会为这件事的四个不同部分各雇一位专员,前提是他们彼此之间不需要交流吗?"如果答案是"会",编排者模式正合适;如果答案是"不会"——因为这些专员需要持续协调——那这件事就该交给一个 subagent,或者走顺序模式。这个"人类团队"类比,能干净地映射到 subagent 设计;训练自己会问这个类比问题的开发者,很快就能一眼看出哪些 workflow 最受益于并行。

A worked end-to-end orchestration looks like this in practice. The developer types her broad request. The main session, recognizing the orchestratable pattern, identifies the dimensions and dispatches the subagents. The dispatches happen in quick succession, and the subagents begin work simultaneously. The main session, while waiting, sometimes does a small amount of preparatory work, like inspecting the diff itself for the synthesis step. The subagents return their reports as they finish; reports may not arrive in dispatch order, since some specialties take longer than others. The main session collects them, holds them until all are present, and then composes the unified output. The whole interaction takes only marginally longer than the slowest of the four subagents. The developer sees a thorough, multi-dimensional review delivered in roughly the time of one careful pass, and the review reads as if four specialists had been retained for the job, because, in a meaningful sense, four specialists were.

一个"端到端"的编排者流程,实操中是这样的:开发者用宽泛的方式提出请求;主 session 识别出"这是一个可编排的形状",找出涉及的维度,派 subagent;派单一份接一份快速发生,subagent 同时开始工作。主 session 在等的当口,有时会做一点准备性工作——比如亲自扫一眼 diff,方便后面综合。各 subagent 完成的顺序可能和派单顺序不一样,因为有的专长耗时长、有的耗时短;它们一边完成一边把报告送回,主 session 收齐之后,会一直等所有报告都到齐,再合成统一输出。整个交互的耗时,只比四个 subagent 中最慢的那一个多一点点。开发者看到的,是一次彻底、多维的评审——送达的耗时,大致相当于"做一次认真评审"的时间;而读起来,就像是真的请了四位专员——在有意义的层面上,确实就是请了四位。

A concrete example will fix this in the reader's mind. Consider a developer who has just opened a pull request that touches roughly four hundred lines of code in three files. The change introduces a new authentication endpoint. Before merging, she wants a full review. She types into Claude Code, in plain language, perform a full review of this PR covering security, performance, style, and test coverage. The main session reads the request. It recognizes the four named dimensions. It looks at the available subagents and finds matches: security-auditor, performance-reviewer, code-style-checker, test-coverage-analyzer. It dispatches all four with a brief that includes the diff and a directive to focus on the subagent's named dimension.

一个具体例子会让这件事在读者脑子里落地。设想:开发者刚开了一个 PR,跨三个文件、改动约四百行,新增一个认证端点。合并前,她想要一次完整评审。她用日常语言在 Claude Code 里敲:"对这个 PR 做一次完整评审,覆盖安全、性能、风格、测试覆盖。"主 session 读这条请求,识别出里面点名的四个维度,去看可用的 subagent,找到匹配项:security-auditor、performance-reviewer、code-style-checker、test-coverage-analyzer。然后它把四个一起派出去,brief 里都带上 diff,并附一条"聚焦你被点名的那个维度"的指令。

The four subagents begin work simultaneously. The security-auditor reads the new authentication code with attention to OWASP categories. The performance-reviewer scans for hot-path issues, queries inside loops, and unbounded allocation. The code-style-checker reads the diff against the project's style conventions. The test-coverage-analyzer cross-references the diff against existing test files to identify which new code paths are unexercised. Each subagent works in a context that contains only the diff and its own focused system prompt. None of them sees the others' work. Each produces a focused report.

四个 subagent 同时开工。security-auditor 用 OWASP 类目去读新的认证代码;performance-reviewer 扫"热路径问题、循环里嵌查询、无界分配";code-style-checker 把 diff 对照项目的风格规约;test-coverage-analyzer 把 diff 和现有测试文件做交叉对照,找出哪些新增代码路径没有被覆盖到。每个 subagent 的上下文里都只有"diff + 它自己那份聚焦的 system prompt",看不到其他 subagent 在做什么,各自产出一份聚焦的报告。

Roughly two minutes later, the four reports return to the main session in whatever order they finish. The security-auditor flags a missing rate limit on the password reset endpoint and a token expiry that defaults to twenty-four hours rather than the project's standard of one. The performance-reviewer flags nothing significant. The code-style-checker notes two minor style deviations. The test-coverage-analyzer points out that the new password reset endpoint has no test coverage at all. The main session reads all four reports, composes a unified review with sections for each dimension, and presents the result to the developer. Total wall-clock time, from her first prompt to the unified review on her screen, is roughly two and a half minutes. Total human attention required, zero, until the review is ready to read.

大约两分钟后,四份报告按各自的完成顺序回到主 session。security-auditor 标出"密码重置端点没有限流"以及"token 过期默认是 24 小时,而不是项目规范里的 1 小时";performance-reviewer 没有发现重大问题;code-style-checker 指出两处轻微的风格偏差;test-coverage-analyzer 指出"新的密码重置端点完全没有测试覆盖"。主 session 读完四份报告,把它们编成一份统一评审——按维度分节,呈现给开发者。从她敲下第一条 prompt,到屏幕上的统一评审出现,挂钟时间总共大约 2.5 分钟。她要付的注意力,是零——直到评审可以读了为止。

Without the Pattern · 没有这个模式会怎样

Now imagine the same review without the orchestrator pattern. The developer would have asked the main session to do the review directly. The main session would have read the diff, then mentally switched into security mode, then into performance mode, then into style mode, then into

test-coverage mode, all in a context that accumulated the residue of each phase. The total wall-clock time would have been closer to ten minutes, because the work was sequential rather than parallel. The total quality would have been lower, because each phase was attended to by a generalist rather than a specialist. And the cost in tokens, while lower, would have been a false economy, because the developer's attention is more valuable than the marginal token spend, and the orchestrated review let her get to her next task seven minutes sooner.

现在想象一下:同一份评审,如果不用编排者模式,会怎样。开发者会直接让主 session 做这件事。主 session 会先读 diff,然后在脑子里"切到"安全模式,再切到性能模式,再切到风格模式,再切到测试覆盖模式——所有这一切,都发生在一个不断累积"每一阶段残渣"的上下文里。总挂钟时间会更接近十分钟,因为是顺序而不是并行;总质量会更低,因为每一阶段都只由一个通才在分心兼顾;token 成本确实更低,但那是一种"虚假的节约"——开发者的注意力比那点边际 token 成本更值钱,而且编排式评审让她提前七分钟进入下一件事。

← 前一页

后一页 →

The orchestrator pattern is not the only multi-subagent pattern, although it is the most visually striking one. A second important pattern is the sequential pipeline, in which the output of one subagent becomes the brief for the next. This pattern applies when the work has a real ordering dependency that cannot be parallelized. A common example: the developer wants to refactor a substantial piece of code, but only after having an architectural review of the current structure to confirm the refactor's direction. The architecture-reviewer runs first; its output, a recommendation about which structure is appropriate, becomes part of the brief for the refactor-specialist that runs second. The two cannot run in parallel because the second one needs the first one's conclusions.

编排者模式不是唯一的"多 subagent 模式",虽然它最显眼。第二个重要模式是**顺序管线(sequential pipeline)**——上一个 subagent 的输出,成为下一个 subagent 的 brief。这种模式适合那种"工作有真实顺序依赖、无法并行"的情形。一个常见例子:开发者想重构一大块代码,但她希望先有"对当前结构的架构评审"来确认重构方向。architecture-reviewer 先跑,它的输出——"推荐走哪种结构"——成为 refactor-specialist 第二个跑的 brief 的组成部分。两者不能并行,因为第二个的输入是第一个的结论。

The mechanics of sequential pipelines are slightly different from those of parallel orchestration. The main session dispatches the first subagent, waits for its result, integrates the result into the brief for the next subagent, dispatches that one, waits for its result, and so on. The total wall-clock time is the sum of each subagent's time, with no parallelism saving. The benefits are still substantial: each step happens in a focused context, each step's output is shaped by the previous step's output rather than by the noise of an accumulated session, and the pipeline can be reviewed step by step, with the option to stop at any intermediate stage if the work is not going where the developer wants. Sequential pipelines, while slower than orchestrations, are the natural shape of work that has real dependencies, and the developer who recognizes when she is in a sequential rather than a parallel situation will save herself the frustration of trying to force orchestration onto work that does not support it.

顺序管线和并行编排的"机械结构"略有不同。主 session 先派第一个 subagent,等它的结果,把结果整合进下一个 subagent 的 brief,再派出那一个,等它的结果,如此往复。总挂钟时间是各 subagent 时间之和,没有"并行带来的节省"。但它带来的好处仍然不小:每一步都跑在一个聚焦的上下文里;每一步的输出由上一步的输出塑造,而不是被一个"不断累积的 session 噪声"塑造;管线可以一步一步地被审视,任何中间阶段,如果工作偏离了开发者的预期,都有机会停下。顺序管线虽然比并行编排慢,但它正是"工作真有依赖"时的天然形状;能分辨"我这是顺序而非并行"的人,会免去"硬把不该并行的东西套上编排模式"的挫败感。

A third pattern, which is a hybrid of the first two, is worth naming briefly. The map-reduce pattern dispatches several subagents in parallel on independent components of a task and then dispatches a final subagent, sequentially, to synthesize their outputs. Sometimes the synthesis is straightforward enough for the main session to do directly, in which case it is just the orchestrator pattern. Sometimes the synthesis is itself a substantial specialty, such as composing a coherent design document from several focused reports, in which case a synthesis subagent is worth

defining. The map-reduce pattern shows up most often in larger codebases, where the inputs are too varied for a single specialist to read in full. Different parallel subagents read different sections; a synthesis subagent receives their reports and produces the final output. The pattern is more elaborate, but the underlying principles are the same: independent work in parallel where possible, dependent work in sequence where required, focused contexts at every stage.

第三种模式,介于前两种之间,值得简短给它起个名字:**map-reduce 模式**。它先并行派几个 subagent,各自处理任务里独立的部分,然后再(顺序地)派一个最后的 subagent 去综合它们的输出。有时综合本身简单到主 session 直接做就行,那就还是编排者模式;有时综合本身就是一门分量不轻的专长——比如把几份聚焦的报告编成一份连贯的设计文档——这时候就值得专门定义一个"synthesis" subagent。map-reduce 模式在大代码库里出现得最多——那里输入过于多样,一个专员读不过来:并行 subagent 各自读自己那一段,synthesis subagent 接住它们的报告,产出最终输出。模式本身更复杂,但底层原则没变:能并行就并行,有依赖就顺序,每一阶段都跑在聚焦的上下文里。

← 前一页

后一页 →

A short reflection on what makes orchestration genuinely powerful. The most striking thing about the orchestrator pattern is not the wall-clock saving, although the saving is real and welcome. The most striking thing is what the developer is freed from. She no longer has to be the one who remembers all the dimensions a thorough review should cover. The pattern itself remembers them. She no longer has to mentally context-switch between security thinking and performance thinking and style thinking, because the subagents handle each dimension in its own thread. Her attention, which was previously distributed across all the dimensions she cared about, can be concentrated on whatever the unified output most demands of her. She becomes, in effect, a senior engineer reading reports written by competent specialists, rather than a junior engineer trying to be all four specialists at once. The transformation in working experience is not just a productivity improvement; it is a change in the kind of cognitive work she is asked to do, and developers who experience it for the first time often describe the shift as the moment Claude Code stopped feeling like a chat interface and started feeling like an environment.

简短聊一下"编排者模式真正强大的地方"。它最显眼的地方,不是挂钟时间的节省——虽然那也是真实且受欢迎的。它最显眼的地方,是开发者被解放出来。她不再需要亲自去记"一次彻底评审该覆盖哪些维度"——模式本身替她记着;她不再需要在脑子里从"安全思维"切到"性能思维"再切到"风格思维"——subagent 各自在自己的线程里处理各自的维度。她原本被摊薄在"所有她关心的维度"上的注意力,可以集中到统一输出最需要她投入的地方。她在效果上变成了"一位资深工程师,读着由胜任的专员写就的报告",而不是"一位初级工程师,试图一个人扮四角"。这种工作体验上的转变,不只是生产力的提升,更是她被要求去做的那类认知工作本身的改变——很多开发者第一次体会到这种转变时会这样描述:"就是这一刻,Claude Code 不再像一个聊天界面,而像一个环境。"

When Not to Orchestrate · 什么时候不该编排

A final note, before this chapter closes, on when not to reach for orchestration. The orchestrator pattern carries overhead. Dispatching four subagents and synthesizing their outputs is more elaborate than asking the main session to do a small task directly. For small tasks, the overhead is not worth paying. A quick code review of a fifty-line patch does not need four subagents; the main session can handle it directly. A trivial documentation update does not need a documentation specialist; the main session is fine. The orchestrator pattern earns its keep on substantial tasks, where the parallelism saving and the specialization benefit are worth the dispatch overhead. The developer who reaches for orchestration appropriately, and refrains from reaching for it on tasks that do not justify it, will have the best experience.

本章收尾前,还要说一点:什么时候不该用编排。编排者模式本身有开销——派四个 subagent、再综合它们的输出,比"让主 session 直接做一件小事"要复杂得多。对小事来说,这笔开销不值得付。五十行 patch 的快速评审,不需要四个 subagent,主 session 直接做就够;琐

碎的文档更新,不需要一个文档专员,主 session 自己就行。编排者模式只有在大体量任务上才回本——那里并行的节省和专门化的好处能压过派单开销。能"该用就用、不该用就克制"的人,体验是最好的。

A useful rule of thumb is the five-minute test. If the developer would spend more than five minutes on the task as a single sequential pass through the main session, the orchestrator pattern is probably worth considering. If she would spend less than five minutes, orchestration is probably overkill. The threshold is not exact, and individual judgment varies, but the rule prevents both common mistakes: orchestrating tiny tasks where the dispatch overhead dominates, and serializing large tasks where the parallelism would have produced real savings. With practice, the developer stops needing the rule, because the right shape of work for orchestration becomes visible at a glance. Until that intuition has set in, the five-minute test is a reasonable working approximation.

一条好用的经验法则叫"五分钟测试"。如果开发者自己一个人顺序做这件事要花超过五分钟,那么编排者模式大概值得考虑;如果只要不到五分钟,编排大概率是杀鸡用牛刀。门槛不是死数,个人判断也会有出入,但这条规则可以同时防止两种常见错误——在小任务上"派单开销盖过收益"、在大任务上"本可并行的却被迫串行"。练上一阵之后,这条规则就不必再挂着——编排适合的形状会"一眼可见"。在那种直觉还没稳之前,"五分钟测试"是一个合理的工作近似。

← 前一页

后一页 →

A second observation worth keeping in mind is that orchestration scales with the developer's familiarity. New users of subagents tend to under-orchestrate, treating the main session as the default and reaching for subagents only on the most obvious cases. Experienced users tend to orchestrate naturally, recognizing the orchestratable shape of work in their first or second pass through a problem. The transition from the first state to the second takes only a few weeks of deliberate practice, and the practice is enjoyable rather than tedious, because every successful orchestration produces a noticeable, satisfying acceleration of work that would otherwise have been slow.

第二条值得记在心里的观察是:编排这种"能力"会随着开发者的熟练度而增长。新接触 subagent 的用户倾向于"编排不足"——把主 session 当默认,只在最明显的几种情况里伸手去用 subagent;熟练用户则会自然地编排,在面对一个问题的第一遍、第二遍里就识别出"可编排"的形状。从前者到后者的过渡,只需要几周有意识的练习;而这种练习并不令人疲倦,因为每一次成功的编排都会带来"原本很慢的事被显著加速"那种可感知的满足。

One final reflection on this chapter, before moving to the inventory of specialists. The orchestrator pattern is the visible face of subagents, the part that produces the most striking demonstrations and the most quotable speedups. But the deeper value of subagents, the value that compounds quietly across months of use, is not the speed. It is the consistent quality. A library of well-designed specialists produces consistently high-quality work because each specialist has been designed for its specific role, and the design is preserved across every invocation. The developer who has installed a serious subagent library is not just faster. She is more reliably good at her work, because the failure modes that come from overload, distraction, or context pollution have been engineered out of her environment. The reliability is the prize. The speed is a happy side effect.

在过渡到"专员清单"那一章之前,再留一段反思。编排者模式是 subagent 的"面子"——它造就了最抓眼球的演示、最值得引用的加速。但 subagent 更深层的价值——那种"在几个月里悄悄复利"的价值——不是速度,而是始终如一的质量。一套精心设计的专员库能稳定地产出高质量的工作,因为每位专员都是为它那个特定角色设计的,而这份设计在每一次调用里都被保留下来。装下了一套"像样的 subagent 库"的开发者,不只是更快——她更稳定地做得好;因为"过载、分心、上下文污染"带来的失败模式,已经从她的环境里被工程化地排除了。可靠性才是奖赏;速度只是顺带的好处。

The next chapter shifts from the single orchestration pattern to the inventory question. With the design of single subagents and the architecture of parallel orchestration both established, the developer needs a working library of subagents to draw from. The chapter ahead provides that library. Eight specialist subagents, each ready to install, each addressing a recurring need in daily development work, will make the architecture of this part of the book concrete in installed form. By the end of the next chapter, the reader will have a team of specialists waiting in her `.claude/agents` directory, ready to be dispatched whenever her work calls for them.

下一章从"单个编排模式"切换到"清单"问题。当"单个 subagent 的设计"和"并行编排的架构"都已经确立,开发者需要一套"能上手就用"的 subagent 库。下一章就提供这套库:八位专员 subagent,每位都已经准备好装,每位都对应一项日常工作里的高频需求——把本部分讲过的架构,以"已安装形态"具体化。等读完下一章,读者就会在自己 `.claude/agents` 目录里坐拥一支"专员团队",随时等候工作召唤。

A worked end-to-end orchestration looks like this in practice. The developer types her broad request. The main session, recognizing the orchestratable pattern, identifies the dimensions and dispatches the subagents. The dispatches happen in quick succession, and the subagents begin work simultaneously. The main session, while waiting, sometimes does a small amount of preparatory work, like inspecting the diff itself for the synthesis step. The subagents return their reports as they finish; reports may not arrive in dispatch order, since some specialties take longer than others. The main session collects them, holds them until all are present, and then composes the unified output. The whole interaction takes only marginally longer than the slowest of the four subagents. The developer sees a thorough, multi-dimensional review delivered in roughly the time of one careful pass, and the review reads as if four specialists had been retained for the job, because, in a meaningful sense, four specialists were.

一个"端到端"的编排者流程,实操中是这样的:开发者用宽泛的方式提出请求;主 session 识别出"这是一个可编排的形状",找出涉及的维度,派 subagent;派单一份接一份快速发生,subagent 同时开始工作。主 session 在等的当口,有时会做一点准备性工作——比如亲自扫一眼 diff,方便后面综合。各 subagent 完成的顺序可能和派单顺序不一样,因为有的专长耗时长、有的耗时短;它们一边完成一边把报告送回,主 session 收齐之后,会一直等所有报告都到齐,再合成统一输出。整个交互的耗时,只比四个 subagent 中最慢的那一个多一点点。开发者看到的,是一次彻底、多维的评审——送达的耗时,大致相当于"做一次认真评审"的时间;而读起来,就像是真的请了四位专员——在有意义的层面上,确实就是请了四位。

A concrete example will fix this in the reader's mind. Consider a developer who has just opened a pull request that touches roughly four hundred lines of code in three files. The change introduces a new authentication endpoint. Before merging, she wants a full review. She types into Claude Code, in plain language, perform a full review of this PR covering security, performance, style, and test coverage. The main session reads the request. It recognizes the four named dimensions. It looks at the available subagents and finds matches: security-auditor, performance-reviewer, code-style-checker, test-coverage-analyzer. It dispatches all four with a brief that includes the diff and a directive to focus on the subagent's named dimension.

一个具体例子会让这件事在读者脑子里落地。设想:开发者刚开了一个 PR,跨三个文件、改动约四百行,新增一个认证端点。合并前,她想要一次完整评审。她用日常语言在 Claude Code 里敲:"对这个 PR 做一次完整评审,覆盖安全、性能、风格、测试覆盖。"主 session 读这条请求,识别出里面点名的四个维度,去看可用的 subagent,找到匹配项:security-auditor、performance-reviewer、code-style-checker、test-coverage-analyzer。然后它把四个一起派出去,brief 里都带上 diff,并附一条"聚焦你被点名的那个维度"的指令。

The library is now eight specialists strong. The reader who has installed all of them has, in roughly an hour of work, assembled a small but capable team that can be dispatched on demand whenever her work crosses one of their domains. The team is patient. It does not get tired. It does not have meetings. Each specialist is excellent at one thing and confidently silent about everything else. The developer's job is to be the editor, dispatching the right specialist to the right work and synthesizing the results into the unified outputs her real life demands.

到这里,你的 subagent 库就有八位专员了。装齐了八位的读者,大约花了一小时的工作,就已经搭起了一支"小而能干"的团队——只要工作触碰到它们的专长领域,随叫随到。这支团队有耐心,不会累,不开会;每位专员都精于一件事,并对其余的事坦荡地沉默。开发者的角色,是编辑——把对路的专员派去对路的工作上,把它们的产出综合成"她真实生活里需要"的统一形态。

The next chapter shifts from the single orchestration pattern to the inventory question. With the design of single subagents and the architecture of parallel orchestration both established, the developer needs a working library of subagents to draw from. The chapter ahead provides that library. Eight specialist subagents, each ready to install, each addressing a recurring need in daily development work, will make the architecture of this part of the book concrete in installed form. By the end of the next chapter, the reader will have a team of specialists waiting in her `.claude/agents` directory, ready to be dispatched whenever her work calls for them.

下一章从"单个编排模式"切换到"清单"问题。当"单个 subagent 的设计"和"并行编排的架构"都已经确立,开发者需要一套"能上手就用"的 subagent 库。下一章就提供这套库:八位专员 subagent,每位都已经准备好装,每位都对应一项日常工作里的高频需求——把本部分讲过的架构,以"已安装形态"具体化。等读完下一章,读者就会在自己 `.claude/agents` 目录里坐拥一支"专员团队",随时等候工作召唤。

← 前一页

后一页 →

CHAPTER THIRTEEN · 第十三章

The Subagent Library: Eight Specialists Every Developer Needs

子代理库:每位开发者都需要的八位专员

"A small team of true specialists outperforms a large team of competent generalists almost every time. The difference is not effort. It is depth."

— attributed to W. Edwards Deming

"一支由真正专员组成的小团队,几乎每一次都胜过一支由"称职的通才"组成的大团队。差距不是投入的多少,而是深度。"

—— 归于 W. 爱德华兹·戴明

This chapter is the practical counterpart to Chapter Nine. By the time the reader finishes it, eight specialist subagents will be installed in her `~/ .claude/agents` directory, ready for use in any project. Each specialist is presented in the same uniform format: the problem it solves, the complete subagent file, and a short explanation of the design choices that shaped it. The reader who installs all eight will have a small but capable team of specialists at her disposal, available for dispatch whenever her work crosses one of their domains.

本章是第九章的"subagent 版姊妹篇"。等读者读完后,八个专员 subagent 就会装进她 `~/ .claude/agents` 目录,随时可以在任何项目里使用。每位专员都用统一的格式呈现:它解决的问题、完整的 subagent 文件、以及"塑造它的设计选择"的简短说明。把八位都装齐的读者,会拥有一支"小而能干"的专员团队,每当工作触碰到它们的领域,即可派单。

The selection of eight specialists is deliberate. The eight cover the most common dimensions of code work that benefit from isolation and specialization: security, performance, testing, documentation, architecture, refactoring, dependency hygiene, and migration. These are the dimensions that, in a real engineering organization, would be staffed by named specialists with named roles. Replicating those specialist roles in subagent form is what gives a single developer, working in Claude Code, access to expertise that would otherwise be available only inside a substantial engineering team.

这八位专员的挑选是刻意的。它们覆盖了"代码工作中受益于隔离与专门化"的最常见维度:安全、性能、测试、文档、架构、重构、依赖卫生、迁移。这些维度,在真实工程组织里,会由"有名有姓、有明确角色"的专员来承担。把这些专员角色以 subagent 形式复制出来,正是"让一位开发者、在 Claude Code 里、获得本来只有大型工程团队里才有的那种专业能力"的关键。

Before beginning the installation, a small piece of housekeeping. All eight subagents below are personal subagents, installed at the user level, and therefore live in `~/ .claude/agents`. Each one is a single Markdown file named after the subagent. After installing all eight and restarting Claude Code, the subagents will be available in every session, in every project the developer opens. Project-specific specialists, if needed, can be added later in the `.claude/agents` directory of specific projects.

动手前,先做点内务。下面八位 subagent 都是"个人 subagent",按用户级别安装,因此都放在 `~/ .claude/agents` 里。每个 subagent 就是单个 Markdown 文件,文件名与 subagent 同名。装齐八位、重启 Claude Code 之后,这些 subagent 在每个 session、每个项目里都可用。如果以后需要"项目级"专员,可以在具体项目的 `.claude/agents` 下再加。

The First Specialist · 第一位专员

The first specialist is `security-auditor`, presented in full in the previous chapter. Its job is to audit code for security vulnerabilities. If the reader has not already installed it, the file from Chapter Eleven can be used directly.

第一位专员是 `security-auditor` ——上一章已经完整给出过。它的职责是审计代码里的安全漏洞。如果读者还没装,直接用第十一章的那个文件即可。

The second specialist is `performance-reviewer`. Its job is to review code for performance issues, with a focus on the kinds of problems that show up in production rather than the kinds that show up in micro-benchmarks.

第二位专员是 `performance-reviewer` 。它的职责是评审代码里的性能问题,关注的是"会在生产里表现出来的"那一类问题,而不是"只在微基准里才看得见"的那一类。

```
---
name: performance-reviewer
description: Reviews code for performance issues that affect real-world execution, including algorithmic complexity,
tools: Read, Grep, Glob, Bash
model: sonnet
---

You are a senior performance engineer. Your job is to find performance issues that will affect production behavior,

Your areas of focus:
- Algorithmic complexity ( $O(n^2)$  where  $O(n)$  would do)
- Database queries inside loops (the classic  $N+1$  pattern)
- Excessive memory allocation in hot paths
- Synchronous I/O where async would be appropriate
- Unbounded work that should be paginated or streamed
- Cache misses where caching would clearly help
- Repeated computation that should be memoized

Your method:
1. Read the code you are given.
2. Identify hot paths: code that runs frequently, runs on every request, or runs in tight loops.
3. Apply your areas of focus to those hot paths first. Apply them lightly to cold paths only if something obvious st
4. For each issue, produce a short report with severity, location, description, and recommendation.

What you do not do:
- You do not modify code. You report.
- You do not flag micro-optimizations. A nanosecond saved in a path that runs once a day is not worth a comment.
- You do not pad. If the code looks fine, say so plainly.
- You do not benchmark. Your review is qualitative, based on patterns. The developer can benchmark if she wants quan

End with a one-sentence judgment: clear, minor issues only, or significant issues found.
```

← 前一页

后一页 →

The third specialist is `test-writer` . Its job is to write tests for a given function or module, following the conventions of the project it is operating in.

第三位专员是 `test-writer` 。它的职责是:为给定的函数或模块写测试,遵循它当前所在项目的测试规约。

```
---
name: test-writer
description: Writes test files for a given function, module, or feature, following the test framework and convention
tools: Read, Write, Bash, Glob
model: sonnet
---

You are a senior test engineer. Your job is to write tests that catch real bugs, not tests that only exercise the ha

Your method:
```


1. Read the file under test and any closely related files needed to understand the code's behavior.
2. Detect the test framework in use by inspecting package.json or the project's equivalent. Default to whatever the
3. Identify the public surface of the file: functions, methods, classes that other code calls.
4. For each item on the public surface, write tests covering at minimum:
 - The happy path
 - At least one boundary condition
 - At least one error or invalid-input case
5. Mock external dependencies. Do not make network calls or write to real files in tests.
6. Place test files alongside the code under test, following the project's existing layout.

What you do not do:

- You do not modify the code under test, even if you discover bugs while writing tests. Note the bug as a comment in
- You do not write tests that only call the function and assert that it does not throw. That is not a test.
- You do not mock the function under test. You mock its dependencies.
- You do not produce an excess of nearly-identical tests. Three meaningful tests are better than ten redundant ones.

End with a list of the test files you wrote and any bugs or concerns you noticed in the code under test.

The Documentation Writer · 文档写作者

The fourth specialist is `documentation-writer`. Its job is to write or update documentation for a given module, API, or feature. The specialist is read- and-write within documentation files only, leaving all code untouched.

第四位专员是 `documentation-writer`。它的职责是:为给定的模块、API 或特性写或更新文档。这位专员只在"文档文件"内可读可写,所有代码一律不碰。

← 前一页

后一页 →

The fifth specialist is `architecture-reviewer`. Its job is to look at proposed changes from a step-back perspective, asking whether the structure of the solution is right rather than whether the implementation is correct.

第五位专员是 `architecture-reviewer`。它的职责是:从"后退一步"的视角看被提议的改动,问"解决方案的结构对不对",而不是"实现正确不正确"。

```
---
name: architecture-reviewer
description: Reviews code or proposed changes for architectural concerns: separation of responsibilities, layering v
tools: Read, Grep, Glob
model: opus
---
```

You are a senior software architect. Your job is to look at code from a step-back perspective: is the structure right

Your areas of focus:

- Separation of concerns: does each module have a clear, single responsibility?
- Layering: are higher-level modules depending on abstractions, with implementation details pushed downward?
- Coupling: do modules know about each other only through narrow, intentional interfaces?
- Abstraction quality: are abstractions earning their place, or are they obscuring rather than clarifying?
- Consistency: does this code fit with the patterns already established in the project, or does it introduce a new p

Your method:

1. Read the code in question.
2. Read enough of the surrounding code to understand the existing patterns and conventions.
3. Apply your areas of focus, prioritizing the ones most relevant to this change.
4. Produce a short report. Lead with the most important architectural observation. Do not dilute it with minor stylistic changes.

What you do not do:

- You do not nitpick. Architectural review is about the shape of the solution, not the details of the implementation.
- You do not suggest sweeping rewrites unless the code clearly cannot be evolved into something acceptable.
- You do not modify code.
- You do not pad. Architectural review is qualitative; one good observation is worth a hundred mediocre ones.

End with a one-sentence judgment: structurally sound, structurally questionable but acceptable, or structurally problematic.

← 前一页

后一页 →

The sixth specialist is `refactor-specialist`. Its job is to take working code and produce a refactor that improves a specific named property, without changing behavior.

第六位专员是 `refactor-specialist`。它的职责是:拿到一段"能跑"的代码,产出一个"针对某项具体属性"的重构——并且不改行为。

```
---
name: refactor-specialist
description: Produces a refactor of given code that improves a specified property (readability, testability, cohesion)
tools: Read, Edit
model: sonnet
---
```

You are a senior refactoring engineer. Your job is to produce refactors that make code better along a specific dimension.

Your discipline:

- Behavior preservation is absolute. The refactored code must produce identical observable output for all inputs the original code could produce.
- Improvements are along a single named dimension at a time. If the user asks for readability, you improve readability.
- You explain the change. After producing the refactor, you list what improved and why.

Your method:

1. Read the code carefully.
2. Confirm the dimension you are optimizing for. If unclear, ask the user before proceeding.
3. Identify the highest-leverage opportunity to improve along that dimension.
4. Produce the refactor as a complete replacement of the original, not as a diff.
5. Explain what changed and why, in three to five lines.

What you do not do:

- You do not change behavior. If you notice a bug, you mention it separately and do not silently fix it.
- You do not bundle unrelated improvements into the same refactor.
- You do not introduce new dependencies, frameworks, or libraries unless the user explicitly authorizes them.
- You do not make stylistic changes that do not serve the named dimension.

End with the refactor and the explanation, and nothing else.

The seventh specialist is `dependency-analyzer` . Its job is to inspect the project's dependency manifest and report on any dependencies that are outdated, vulnerable, unused, or duplicated.

第七位专员是 `dependency-analyzer` 。它的职责是:检查项目的依赖清单(manifest),报告"过时、有漏洞、未使用、或重复"的依赖项。

```
---
name: dependency-analyzer
description: Analyzes the project's dependencies for outdated versions, known vulnerabilities, unused packages, and
tools: Read, Grep, Glob, Bash
model: sonnet
---
You are a senior dependency hygiene engineer. Your job is to keep a project's dependency surface clean, secure, and

Your method:
1. Read the dependency manifest (package.json, requirements.txt, Cargo.toml, etc.).
2. Read the lockfile if one exists.
3. Identify, in order of priority:
- Dependencies with known security vulnerabilities (run the project's audit tool if available)
- Major-version updates available for actively maintained dependencies
- Dependencies that appear in the manifest but are not imported anywhere in the source
- Duplicate or redundant dependencies (two libraries doing the same job)
- Dependencies with significant licensing concerns
4. Produce a report grouped by category, with each item including the package name, current version, latest version

What you do not do:
- You do not update the manifest. You only report.
- You do not flag every outdated minor version. Focus on major-version drift and security concerns.
- You do not invent vulnerabilities. If your audit tool produces no findings, say so plainly.
- You do not pad. A clean dependency tree gets a one-line judgment, not a pseudo-report.

End with a one-sentence judgment of overall dependency health: clean, minor concerns only, or significant cleanup re
```

The eighth and final specialist is `migration-planner` . Its job is to produce a step-by-step plan for a non-trivial migration, such as moving from one framework to another, upgrading a major dependency, or restructuring a substantial part of a codebase.

第八位、也是最后一位专员是 `migration-planner` 。它的职责是:为一次"非平凡"的迁移产出一份"分步骤"的方案——比如从一个框架换到另一个、升级一个主版本依赖、或重构代码库里的相当一部分。

```

---
name: migration-planner
description: Produces a step-by-step migration plan for a substantial change such as a framework upgrade, language v
tools: Read, Grep, Glob
model: opus
---
You are a senior migration engineer. Your job is to break a substantial migration into a sequence of safe, reviewabl

Your principles:
- A good migration plan is a sequence of small, ship-able changes that can each be reviewed and reverted independent
- Every step has a clear acceptance criterion. The team should know when each step is complete and whether it succee
- Risk goes at the front of the plan, not the back. Identify the riskiest part of the migration and propose a small
- Honest plans assume some steps will fail. Include a rollback strategy for each non-trivial step.

Your method:
1. Read the relevant code to understand the current state.
2. Read enough about the target state to understand the destination.
3. Decompose the migration into a numbered sequence of steps. Aim for steps that take a single PR each, no more.
4. For each step, document: what changes, what does not change, the acceptance criterion, the rollback strategy, and
5. Identify the riskiest step and propose an early experiment that reduces the risk before the team commits to the f
6. End with a one-paragraph executive summary suitable for a team meeting.

What you do not do:
- You do not write migration code. You produce the plan.
- You do not commit to specific dates. Effort estimates in person-days are appropriate; calendar dates depend on tea
- You do not pad with general advice about migrations. Your output is specific to this migration.

End with the executive summary.

```

← 前一页

后一页 →

The library is now eight specialists strong. The reader who has installed all of them has, in roughly an hour of work, assembled a small but capable team that can be dispatched on demand whenever her work crosses one of their domains. The team is patient. It does not get tired. It does not have meetings. Each specialist is excellent at one thing and confidently silent about everything else. The developer's job is to be the editor, dispatching the right specialist to the right work and synthesizing the results into the unified outputs her real life demands.

到这里,你的 subagent 库就有八位专员了。装齐了八位的读者,大约花了一小时的工作,就已经搭起了一支"小而能干"的团队——只要工作触碰到它们的专长领域,随叫随到。这支团队有耐心,不会累,不开会;每位专员都精于一件事,并对其余的事坦荡地沉默。开发者的角色,是编辑——把对路的专员派去对路的工作上,把它们的产出综合成"她真实生活里需要"的统一形态。

Extending the Library · 扩展你的库

A note on extension. The eight specialists above are starting points, not endpoints. As the developer lives with them, she will discover that her work has shapes the eight do not cover, or that one of the eight is slightly off-fit for her specific stack. The fix is the same as for skills: edit the file. Each subagent is a single Markdown file. Editing it takes thirty seconds. Adjusting the role definition, expanding the description, narrowing the tools, refining the boundary, all of these are small changes that the developer can make as her understanding of her own workflow deepens.

关于"扩展"几句话。上面八位专员,同样是"起点而非终点"。开发者真正用起来之后,会发现:她工作里有"八位都没覆盖"的形状,或者八位中有"稍微不太贴合自己那套技术栈"的。修法与 skill 一样——改文件。每个 subagent 就是单个 Markdown 文件,改它也就三十秒:调整角色定义、扩展 description、收窄工具、打磨边界——这些小改动,开发者可以随着对自己工作流理解的加深,慢慢去做。

A second note on adding new specialists. The developer who finds, after a month or two of using the eight specialists, that a recurring kind of work is not covered by any of them, has the raw material for a ninth specialist. The pattern by which she should design it is by now familiar: name the role specifically, narrow the tools to what the role needs, write boundary language that stops the role from drifting, and test the result on three representative inputs. Within an hour, she has a ninth specialist on the team, and her library has grown along the contour of her actual work.

关于"加新专员"再说几句。开发者如果用了一两个月八位专员之后,发现有一类反复出现的工作不在它们的覆盖范围内——那她就有了"第九位专员"的原材料。设计它的套路到这时已经熟:具体地命名角色、把工具收到它"恰好需要"、写一段边界语言防止它漂移、并在三个有代表性的输入上测试。一小时之内,她就有了第九位专员,库也沿着"自己实际工作"的轮廓继续生长。

← 前一页

后一页 →

A third note on team-level subagents. Several of the specialists above will be useful to other developers in the reader's team or company. The mechanism for sharing is the same as for skills: copy the subagent file into the project's `.claude/agents` directory and commit it. From that point forward, every developer who clones the project receives the specialist automatically. This is how subagent libraries spread within organizations, and how a thoughtful library built by one developer becomes a standing resource for an entire team. The library committed to a project repository is a quiet but real form of leverage. It travels with the code, it activates on the first session, and it raises the floor of what every developer working in that project can produce.

关于"团队级 subagent"再说几句。上面这些专员里,有几位对读者所在团队或公司的其他开发者也会很有用。分享机制与 skill 一样:把 subagent 文件拷进项目的 `.claude/agents`,提交进版本库。从那之后,凡是 clone 这个项目的开发者,都会自动拿到这位专员。subagent 库在组织内就是以这种方式传播的——"一位开发者精心搭出的库"会变成"整支团队的一份常驻资源"。提交进项目仓库的 subagent 库,是一种静默但真实的杠杆:它随代码一起走,在第一个 session 就激活,会拉高"在那个项目里工作的每一位开发者"所能产出的底线。

The chapter that follows is the climax of Part Three. With skills installed and subagents installed, the question becomes how the two systems compose. Subagents can use skills. Skills can be designed to fit specific subagents. The combination is where the architectural depth of Claude Code becomes most visible, and the next chapter is dedicated entirely to it. The reader who has installed both libraries by the end of Part Two and Part Three is now in possession of the components for a substantial leap in capability. The next chapter shows how to assemble those components into the integrated system that has been the destination of the book all along.

下一章是第三部分的高潮。Skill 装好、subagent 也装好之后,问题就变成了"这两套系统怎么组合在一起"。Subagent 可以使用 skill;skill 也可以被设计来"配"某个 subagent。两者的组合,正是 Claude Code 架构深度最显形的地方——下一章就专门讲这个。等第二、第三部分都读完,两个库都装上的读者,就已经手握足以"让能力跃迁一档"的组件;下一章,就是把它们组装成"这本整书一直通向的那座集成系统"。

A note on extension. The eight specialists above are starting points, not endpoints. As the developer lives with them, she will discover that her work has shapes the eight do not cover, or that one of the eight is slightly off-fit for her specific stack. The fix is the same as for skills: edit the file. Each subagent is a single Markdown file. Editing it takes thirty seconds. Adjusting the role definition, expanding the description, narrowing the tools, refining the boundary, all of these are small changes that the developer can make as her understanding of her own workflow deepens.

关于"扩展"几句话。上面八位专员,同样是"起点而非终点"。开发者真正用起来之后,会发现:她工作里有"八位都没覆盖"的形状,或者八位中有"稍微不太贴合自己那套技术栈"的。修法与 skill 一样——改文件。每个 subagent 就是单个 Markdown 文件,改它也就三十秒:调整角色定义、扩展 description、收窄工具、打磨边界——这些小改动,开发者可以随着对自己工作流理解的加深,慢慢去做。

The library is now eight specialists strong. The reader who has installed all of them has, in roughly an hour of work, assembled a small but capable team that can be dispatched on demand whenever her work crosses one of their domains. The team is patient. It does not get tired. It does not have meetings. Each specialist is excellent at one thing and confidently silent about everything else. The developer's job is to be the editor, dispatching the right specialist to the right work and synthesizing the results into the unified outputs her real life demands.

到这里,你的 subagent 库就有八位专员了。装齐了八位的读者,大约花了一小时的工作,就已经搭起了一支"小而能干"的团队——只要工作触碰到它们的专长领域,随叫随到。这支团队有耐心,不会累,不开会;每位专员都精于一件事,并对其余的事坦荡地沉默。开发者的角色,是编辑——把对路的专员派去对路的工作上,把它们的产出综合成"她真实生活里需要"的统一形态。

CHAPTER FOURTEEN · 第十四章

Subagents and Skills: The Composition Pattern

子代理与技能:组合模式

"The whole, when properly composed, contains capabilities that none of its parts possess in isolation. This is the meaning of architecture."

— attributed to a systems engineer, anonymous

"整体,当被恰当地组合起来,会拥有"任一部分单独都不具备"的能力。这就是"架构"这个词的含义。"

—— 归于一位系统工程师(佚名)

This chapter is where the two pillars built so far in the book begin to reinforce each other in a way that produces capabilities neither could produce alone. Skills, the subject of Part Two, taught Claude how to do specific tasks consistently. Subagents, the subject of Part Three, gave Claude a team of specialists, each in its own context. The composition of the two is the architectural depth that the book has been building toward, and this chapter teaches it explicitly.

本章,书里目前搭起的两根支柱开始彼此加固,以一种"任何一根独自都做不到"的方式产出能力。第二部分讲的 Skills,教的是 Claude 如何一致地完成具体任务;第三部分讲的 Subagents,给了 Claude 一支"各自独立、各有上下文"的专员团队。两者的组合,正是这本书一直在朝向的"架构深度"——本章把这件事明白地讲出来。

Begin with the basic mechanic. Skills are available everywhere. They are loaded at the level of the Claude Code installation, and any Claude process running in that installation, including any subagent, can activate any applicable skill. A subagent does not need a special configuration to access skills; it simply needs to have a description that matches the kind of work where a skill would naturally fire. The composition is not bolted on. It is the default behavior of the two systems when they are present together, and the developer's job is to design with the composition in mind from the start.

先说最基本的机制。Skill 是"处处可用"的——它们在 Claude Code 安装层加载,任何跑在该安装里的 Claude 进程(包括任何 subagent)都可以激活任何适用的 skill。subagent 要访问 skill 不需要任何特殊配置;它只需要让自己的 description 与"某个 skill 自然会触发的"那类工作相匹配。这种组合不是"外挂"上去的——它是"两套系统同时存在"时的默认行为;开发者的任务是:从一开始就把"组合"放进设计的考虑里。

A worked example illustrates the power. The developer has a `test-writer` subagent installed, presented in the previous chapter. The developer also has a project-specific skill called `test-conventions`, which encodes how tests are organized and named in this particular codebase, including the choice of test framework, the file layout, the mocking style, the use of fixtures, and the team's preferred patterns for assertion phrasing. The `test-conventions` skill lives in the project's `.claude/skills` directory; the `test-writer` subagent lives in the user's `~/.claude/agents` directory.

一个具体例子能说明这种力量。开发者装好了一个 `test-writer` subagent(上一章给出过)。她还装了一个项目级 skill,叫 `test-conventions` ——它把"这个特定代码库里测试是怎么组织、命名的"编码进去,包括:测试框架的选择、文件布局、mock 风格、fixture 的使用方式、团队偏好的断言措辞。`test-conventions` skill 放在项目的 `.claude/skills` 里; `test-writer` subagent 放在用户的 `~/.claude/agents` 里。

Now imagine the developer dispatches the test-writer subagent on a new function. The subagent receives its brief: write tests for this function. The subagent begins its work. It reads the function. As it considers the structure of the tests it is about to write, it recognizes that the request matches the description of the `test-conventions` skill, and it loads that skill into its own context. The subagent now has, simultaneously: its own role as a senior test engineer, its own focused tools and isolated context, and the project-specific knowledge encoded in the skill. The tests it produces are written in the project's exact conventions, by a specialist who knows how to write good tests in general, in a context that is not polluted by anything outside the test-writing task.

现在想象:开发者把 `test-writer` subagent 派去处理一个新函数。subagent 收到 brief:为这个函数写测试。subagent 开始工作,读了函数。当它琢磨"接下来要写的测试是什么结构"时,识别出这个请求匹配 `test-conventions` skill 的 description,于是把那个 skill 加载进了自己的上下文。此时的 subagent,身上同时有三样东西:它自己"高级测试工程师"的角色、自己聚焦的工具与隔离的上下文、以及 skill 里编码的项目特定知识。它产出的测试——既严格遵循项目规约,又出自一位"在一般意义上知道怎么写好测试"的专员之手,而且跑在一个"没有被测试任务之外的任何事污染"的上下文里。

Composition in Action · 组合在行动

The result is strikingly better than what either component could produce alone. A skill alone, fired in the main session, would shape the output toward project conventions, but the output would be subject to context pollution from whatever else the main session was working on. A subagent alone, dispatched on the same task, would produce well-isolated, generally good tests, but it would not know the project's specific conventions and would have to guess. The combination has both: convention-correct tests, produced in a clean context, by a specialist who treats the work seriously. This is the composition pattern in its simplest form, and once a developer sees it work even once, she will not want to design subagents that do not know how to use skills.

这种结果,比"任何一方单独能产出的"都明显要好。Skill 单独、在主 session 里触发,确实会让输出向项目规约靠拢,但那份输出仍会被主 session 里其他事情的"上下文污染"影响。Subagent 单独、被派去做同一件事,确实能产出"隔离良好、整体不错"的测试,但它并不知道这个项目的具体规约,只能猜。两者的组合同时拥有这两点:规约准确的测试、由"把工作当回事"的专员、在干净的上下文里产出。这就是组合模式最简单的形态;一旦开发者哪怕只看它工作一次,她就再也不想设计"不懂用 skill"的 subagent 了。

There is a design principle that follows from this and that should guide every subagent the developer writes from now on. Subagents should be designed as roles, not as repositories of project knowledge. A test-writer subagent should know how to be a senior test engineer in general; it should not know that the project uses Vitest rather than Jest, or that mocks should be placed in a specific directory, or that the team prefers a specific assertion library. Those are project facts, and they belong in skills. The subagent provides the role; the skill provides the project-specific knowledge. When the role meets the project, the composition produces the right behavior.

由此引出一条设计原则,它应该从今往后指导开发者写的每一个 subagent:subagent 应该被设计成角色,而不是"项目知识的容器"。一个 test-writer subagent 应当知道"如何作为一位高级测试工程师",而不应当知道:这个项目用的是 Vitest 而不是 Jest、mock 要放在某个特定目录、团队偏好某个特定的断言库。这些是项目事实,属于 skill。subagent 提供角色,skill 提供项目特定的知识。当角色与项目相遇,组合就产出正确的行为。

This principle has a second-order effect that is worth naming. A subagent that contains project-specific knowledge becomes specific to that project, and its usefulness drops as soon as the developer moves to a different project. A subagent that contains only the role, with project-specific knowledge factored into skills, is portable. The same test-writer subagent works in five different projects, because the role is the same in each, and the project-specific conventions are loaded as skills from each project's own `.claude/skills` directory. A small library of well-designed, role-only subagents at the user level, combined with project-specific skills in each project, gives the developer the best of both worlds: stable, portable specialists, and project-appropriate behavior in every project she works in.

这条原则有一个值得点名的"二阶效应"。一个把项目特定知识塞在自己体内的 subagent,会变得"只适用于那个项目";开发者一切换到另一个项目,它的价值就立刻下降。一个只装"角色"、把项目特定知识抽到 skill 里的 subagent,才是可移植的。同一份 test-writer subagent,可以在五个不同的项目里都工作得好——因为"角色"在每个项目里都一样,项目特定的规约由"每个项目自己的 `.claude/skills` 目录"以 skill 形态加载进来。一套小而精、只装"角色"的 subagent 库放在用户级,再加上每个项目里"项目级"skill——开发者就同时拿到了两样最好的东西:稳定、可移植的专员,以及"在每个项目里都符合项目实际"的行为。

A second worked example raises the composition to a more sophisticated level. The developer has installed the security-auditor subagent from Chapter Eleven, the architecture-reviewer subagent from Chapter Thirteen, and a project-specific skill called `auth-conventions`, which encodes the codebase's authentication patterns, token handling, and security-sensitive practices. The skill is not project-specific knowledge that the security-auditor needs to do its job in general; the security-auditor knows about OWASP Top 10 and common vulnerability patterns regardless of project. But the skill encodes the way this specific codebase chooses to handle authentication, which is information the security-auditor needs in order to flag deviations from the established pattern as concerns rather than treating them as ordinary alternatives.

第二个具体例子,把组合抬到一个更复杂的层次。开发者装好了第十一章的 `security-auditor` subagent、第十三章的 `architecture-reviewer` subagent,以及一个项目级 skill——`auth-conventions`,把"代码库的认证模式、token 处理、与安全敏感的做法"编码进去。这个 skill 并不是"security-auditor 干它那件一般意义上的事"所必需的项目特定知识——security-auditor 知道 OWASP Top 10、知道常见漏洞模式,跟项目无关;但这个 skill 把"这个特定代码库选择怎么处理认证"编码进去——这正是 security-auditor 需要的:它要把"偏离既定模式"标记成"可疑",而不是当成"普通的备选方案"放过。

When the developer dispatches the security-auditor on a change that touches authentication, two things happen simultaneously. The subagent activates its general security expertise. The subagent also loads the `auth-conventions` skill, because the description matches. The subagent now has both: the general security knowledge that says hardcoded credentials are bad, and the specific project knowledge that says this codebase uses a token-based scheme with refresh rotation handled by a particular module. The audit it produces is doubly informed. It catches general security issues. It also catches deviations from the project's specific patterns, which a generalist would not have known were patterns at all.

当开发者把 security-auditor 派去处理"涉及认证的改动"时,有两件事同时发生:subagent 激活它一般的安全专业知识;同时,因为 description 匹配,subagent 也会加载 `auth-conventions` skill。此时的 subagent 身上,既有"硬编码凭据是不好的"这种一般安全知识,也有"这个代码库使用基于 token 的方案、refresh 轮换由某个特定模块负责"这种项目特定知识。它产出的审计,被双重"喂饱":既能抓出一般安全问题,也能抓出"偏离项目特定模式"的偏差——后者,通才型评审者甚至根本不会知道"原来这里还有种模式"。

An Advanced Composition Pattern · 一种更进阶的组合模式

There is a more advanced composition pattern worth introducing here, although its full development belongs in Part Five. The orchestrator pattern from Chapter Twelve, in which the main session dispatches several subagents in parallel and synthesizes their outputs, can be combined with skill composition. Each parallel subagent can activate its own appropriate skills. A four-way code review dispatched as four parallel subagents, each of which knows how to use the relevant project skills, produces a deeper, more project-aware review than a four-way review in which the subagents operate in their own generic roles without skill support. The combinations compose, and the composition of compositions is where the architecture of Claude Code reaches its most expressive form.

这里值得引入一种更进阶的组合模式——尽管它完整的展开属于第五部分。第十二章的"编排者模式"——主 session 并行派若干 subagent、再综合它们的输出——可以和 skill 组合再组合:每一个并行 subagent 都可以激活它各自合适的 skill。一次"四路"代码评审,作为四个并行 subagent 被派出,且每个 subagent 都知道怎么用相关的项目 skill——它产出的评审,比"四个 subagent 各自跑在通用角色里、没有任何 skill 支撑"的那种四路评审更深、更懂项目。组合之上再组合,正是 Claude Code 的架构抵达其最有表现力形态的地方。

more project-aware review than any single channel could produce. The four-by-four matrix, four subagents and four skills, is the structure of a serious engineering review process, and the composition pattern of this chapter is what makes it a single concept rather than two unrelated ones.

远超任何单一渠道所能产出的"对项目的理解"。这个 4x4 矩阵——四个 subagent 配四个 skill——是真正严肃的工程评审流程的结构;而本章所讲的组合模式,正是把"两个不相关的东西"捏成"同一个概念"的关键。

A practical caveat. Not every subagent benefits from skill composition, and the developer should not over-engineer the design. Some specialists are useful as pure role containers and should not be paired with skills at all. The migration-planner subagent from the previous chapter, for example, produces a generic migration plan that is shaped more by the specifics of the migration than by ambient project conventions. There is little benefit in trying to load skills into it. The composition pattern is a tool, not a doctrine. The developer should reach for it when it produces real value and leave it alone when it would only complicate a workflow that does not need complication.

一个务实的提醒:并不是每个 subagent 都受益于 skill 组合,开发者也不该把设计过度工程化。有些"专家"适合作为纯粹的角色容器存在,根本不该和 skill 配对。比如上一章那个 migration-planner subagent,它产出的迁移计划,主要由"这次迁移本身的具体情况"塑造,而非由"项目环境里那些约定俗成"来定形;硬往里塞 skill 几乎没什么收益。"组合模式"是一个工具,不是教条——它能真正产出价值时再用,只会让本不需要复杂化的流程变复杂时,就别碰。

A second practical caveat concerns description engineering. For the composition to fire correctly, the skill descriptions and the subagent descriptions both need to be written well. The subagent description determines whether the subagent is dispatched. The skill description determines whether the skill activates inside the dispatched subagent. Both layers of triggering need to work for the composition to produce its intended behavior. The same disciplines from Chapter Six apply here: canonical statement, trigger language, boundary, input expectation. The developer who has internalized those disciplines can debug failed compositions quickly. The developer who has not will sometimes find that the composition does not work and will not be sure why.

第二条务实提醒,关于 description 的工程化。要让组合正确触发,skill 的 description 和 subagent 的 description 都得写好。Subagent 的 description 决定它会不会被派出去;skill 的 description 决定它会不会在那个被派出去的 subagent 内部被激活。两层触发都得正常,组合才会产出预期的行为。第六章里那套规矩在这里同样适用:规范化的陈述、触发语、边界、输入预期。已经把那套规矩内化了的开发者,排错时能很快找到失败的组合;没有内化的开发者,有时会面对"组合不工作、却说不清为什么"的窘境。

A third worked example will deepen the pattern. The developer has installed the documentation-writer subagent and a project-specific skill called doc-style-guide, which encodes the team's conventions for technical writing: the preferred section structure, the rules about voice and tense, the format of code examples, and the policy on what level of detail is appropriate for different audiences. The skill is project-specific because every team has its own documentation style, and a documentation-writer subagent that wrote in some generic style would produce documentation that did not fit the team's existing corpus.

第三个完整例子,把模式讲得更透。开发者已装好 documentation-writer subagent,以及一个项目级的 skill,叫 doc-style-guide ——它把团队在技术写作上的全部规约都编码在内:偏好的章节结构、关于"语气/时态"的规则、代码示例的格式、对不同受众该铺多少细节的策略。这个 skill 是项目级的,因为每个团队的文档风格都不一样;一个 documentation-writer subagent 如果产出的是泛泛的"通用风格",那么写出来的文档将和团队既有的整套文档语料不搭。

The composition produces a documentation-writer that is doubly informed. The subagent provides the role: a senior technical writer who knows how to explain code clearly, how to choose examples, and how to keep documentation honest. The skill provides the project conventions: section ordering, voice, tense, example format. When the developer dispatches the subagent on a new module that needs documenting, both fire together. The output reads like good technical writing in the team's house style, because both layers of expertise are present at once. The developer did not have to brief the subagent on the team's conventions, because the skill brought them automatically. She did not have to teach the skill how to write good prose, because the subagent provided that. Each system did its job, and the composition produced output better than either alone.

这一组合,产出的是"双重知情"的 documentation-writer。Subagent 给出"角色":一位资深技术作者,懂得如何把代码讲清楚、如何挑选例子、如何保证文档的诚实。Skill 给出"项目规约":章节顺序、语气、时态、例子格式。当开发者把 subagent 派去写一个待文档化的新模块时,两层同时点火。产出的文字读起来就像团队自家风格里的"好的技术写作"——因为两层专长同时在场。开发者不必再向 subagent 简报团队的规约,skill 自动带来了;也不必教 skill 怎么写出好散文,subagent 那头就准备好了。各自做各自该做的事,而组合的产出,胜过任一者孤军作战的结果。

The Architectural Principle · 架构原则

There is a deeper architectural principle visible here, which is worth naming explicitly because it will guide every later chapter of the book. The composition pattern is an instance of the broader principle of separation of concerns. Skills hold project knowledge. Subagents hold roles and isolate contexts. Neither system tries to do the other's job. When a workflow needs both kinds of capability, the systems compose, and the composition produces capabilities that neither alone could provide. This is the same principle that underlies every well-architected piece of software, applied to the design of a Claude Code environment. Developers familiar with separation of concerns in their own code will recognize the principle immediately and find the composition pattern intuitive. Developers unfamiliar with it will, by working through this part of the book, internalize the principle by seeing it applied to a domain where its benefits are immediate and visible.

这里其实浮现出一条更深的架构原则,值得显式点出来——因为它会指导本书之后每一章。"组合模式"其实是更广义的"关注点分离"原则的一个实例。Skill 持有"项目知识";subagent 持有"角色"并隔离上下文。任一系统都不试图去做对方的工作。当一个 workflow 同时需要这两种能力时,系统就组合起来,而组合的产出,又超出它们各自能独立提供的上限。这与"每一段良好架构的软件"所遵循的是同一条原则,只是被搬到了 Claude Code 环境的设计上来。熟悉"关注点分离"这条原则的开发者,会立刻认出它,并觉得组合模式很自然;不熟悉的开发者,只要走完本书这一部分,也就会在"一个收益立竿见影、看得见摸得着"的领域里把它内化下来。

A practical implication of this principle is the design discipline for adding new capabilities to a Claude Code environment. When the developer notices a recurring kind of work that her current configuration handles poorly, she should ask, is the gap a knowledge gap or a role gap. If she finds herself wishing the environment knew something specific, that is a knowledge gap, and the answer is a skill. If she finds herself wishing a different kind of attention were being applied to a kind of work, that is a role gap, and the answer is a subagent. Sometimes the gap is both, and the

这条原则的一条实务含义,是给 Claude Code 环境"加新能力"时的设计纪律。当开发者注意到有一类反复出现的工作、当前的配置处理得不好时,她应当先问一句:这道缺口,究竟是"知识缺口"还是"角色缺口"?如果她希望"环境知道某件具体的事"——那是知识缺口,答案是写一个 skill。如果她希望"在处理某类工作时,投入的是另一种不同的注意力"——那是角色缺口,答案是写一个 subagent。有时候这道缺口两边都站,那

answer is a composition. Asking the right question first prevents the developer from building skills that should have been subagents, or subagents that should have been skills, both of which are common mistakes that produce environments that work but feel awkward. Asking the right question consistently produces environments that fit the developer's work cleanly, because each capability lives in the right place.

答案是“组合”。先问对问题,就能避免那种“本该写 skill 却被写成了 subagent”或“本该写 subagent 却被写成了 skill”的常见错误——这两种错,都能让环境跑得起来,但用起来总是别扭。每次都问对,就能让每一项能力都“住”在它最该住的地方,环境就能干净地贴合开发者的实际工作。

A second design discipline that emerges from the composition pattern concerns versioning. Skills and subagents that are designed to compose with each other should evolve together. If the developer changes the conventions encoded in a skill, the subagents that depend on that skill may need to be checked to ensure their behavior still aligns. If the developer reshapes a subagent's role, the skills designed to fit that role may need to be revisited. This coupling is loose, not tight, and the systems remain independently functional, but the composition is most reliable when the developer treats related skills and subagents as parts of a single design rather than as unrelated artifacts. The discipline is to version-control the entire `.claude` directory, to commit related changes together, and to review skill and subagent changes side by side when they are likely to affect each other.

从组合模式里浮现出来的第二条设计纪律,关于版本管理。设计上要彼此组合的 skill 与 subagent,应当“共同演进”。如果开发者改了某条 skill 里编码的规约,那些依赖它的 subagent 可能要回头检查一下,看自身行为是否还对得上;如果她重塑了某个 subagent 的角色,那些为该角色“贴身打造”的 skill 也可能需要再审视。这种耦合是“松”而不是“紧”的,系统各自依旧独立可用;但是,当开发者把“相互关联的 skill 与 subagent”当作同一个设计里的若干部件、而不是彼此不相关的零散制品时,组合才最可靠。具体纪律是:把整个 `.claude` 目录纳入版本控制、把相关变更一起提交,在它们可能彼此影响时,把 skill 与 subagent 的变更并排做评审。

One more reflection on the experience of the composition pattern, before this chapter closes. The first time a developer sees a skill activate inside a subagent and watches the combined behavior produce something neither could have produced alone, the experience tends to be memorable. There is a moment of recognition in which the architecture stops feeling abstract and starts feeling like a real engineering substrate. The skill library and the subagent library, which until then had felt like two separate collections of useful artifacts, become something larger. They become the components of an environment, and the environment becomes the place where the developer's real work happens. This shift in perception is not just aesthetic. It is the shift from using Claude Code to engineering with Claude Code, which is the central promise of this book and the moment the entire framework starts paying back the investment of learning it.

在收束本章之前,再就“组合模式的体验”做一点感想。开发者第一次亲眼看见“一个 skill 在某个 subagent 内部被激活”,并看着两者合起来的行为产出了“任何一方单独都产不出”的东西时,这份体验往往是令人难忘的。会有一刻“认出”——架构不再显得抽象,而是开始像一块真正可工程化的底材。原本感觉像“两套互不相干的实用制品”的 skill 库与 subagent 库,忽然变成了某种更大的东西:它们成为一块“环境”的组件;而这块环境,反过来成为开发者真正的工作“发生的地方”。这种认知上的迁移,不只是审美层面的——它正是“从用 Claude Code”到“用 Claude Code 做工程化”的跃迁。这正是本书的核心承诺,也是整副框架开始回报“为学会它所付出的投入”的那一时刻。

A reflection on the architecture, before this chapter and this part of the book end. The design choices that made skills and subagents into separate systems, rather than merging them into a single mechanism, were deliberate. Skills are extensible knowledge. Subagents are specialized roles

在收束本章、也收束本书这一部分之前,再来对架构本身做一点反思。把 skill 与 subagent 设计为两套独立系统、而不是合并成一套机制,是经过深思熟虑的。Skill 是"可扩展的知识";subagent 是"专门化的角色",

with isolated contexts. The two are different kinds of thing, and treating them as different kinds of thing keeps the design clean. A merged system would have been simpler at first glance and worse in practice. The cost of having two systems is the small overhead of learning both. The benefit of having two systems is that each remains focused on what it does best, and the composition between them is principled rather than tangled. The reader who has worked through the last six chapters now has both systems in working memory and in installed form, and she has seen how they reinforce each other to produce capabilities neither could produce alone.

并配以隔离的上下文。两者本质上是不同种类的"东西",把它们当作不同种类的"东西"来对待,设计才能保持清爽。合并的系统初看更简洁,实操起来却更糟。维持两套系统的代价,是同时学两套系统那点小小开销;收益则是,每套系统都聚焦在自己最擅长的事上,二者之间的组合是有章法的、而非缠成一团。走过过去六章的读者,如今两套系统都在手上、也都在已安装的形态里,也已经见过它们如何彼此增益,产出单靠任一者都达不到的能力。

Second-Order Benefits · 二阶收益

There is a second-order benefit to the composition pattern that becomes visible only after a developer has lived with a composed library for a few months. The benefit is that the library becomes self-documenting in a way that monolithic configurations are not. A new developer joining the project can read the `.claude/skills` directory to learn the project's conventions; reading the `SKILL.md` files is essentially reading a structured guide to how the team prefers to work. The same new developer can read the `.claude/agents` directory to learn what specialist roles the team treats as standing capabilities; reading the subagent definitions is essentially reading the team's job descriptions for its AI-assisted workflow. Together, the two directories form a piece of working documentation that is more accurate than any wiki, because the documentation is the configuration the tool actually uses. This self-documenting property is a substantial benefit for any team that grows, because it lowers the cost of onboarding new members and reduces the gap between what the team says it does and what the team actually does.

组合模式还有一条"二阶收益",只在开发者与组合库相伴共处几个月之后才看得清:它让这套库具备一种"自我文档化"的性质,而单体化的配置做不到这一点。新加入项目的开发者,读一读 `.claude/skills` 目录就能学懂项目规约——读那些 `SKILL.md` 文件,本质上就是在读一份"团队偏好如何工作"的结构化指南。同一位新人,再读读 `.claude/agents` 目录,就能了解团队把哪些"专家角色"当作常设能力——读那些 subagent 定义,本质上就是在读"团队 AI 辅助工作流"里的岗位说明书。两个目录合起来,就构成了一份"比任何 wiki 都更准确"的工作文档——因为文档本身,就是工具真正在用的那份配置。对任何会成长的团队,这种自我文档化的属性都是一笔不小的收益:它降低了新成员上手的成本,也缩短了"团队声称在做的事"与"团队实际在做的事"之间的鸿沟。

A third reflection concerns the relationship between this composition and the third pillar of the framework, hooks, which is the subject of the part of the book that follows. The composition of skills and subagents that this chapter has taught is internal to the agent loop. Both skills and subagents operate within the probabilistic space of the language model. They shape what the model does, but they do not impose deterministic constraints on it. Hooks are different. Hooks live outside the loop and impose deterministic constraints on it. The fully engineered Claude Code environment that the book is building toward includes all three pillars working together: skills

第三点反思,关于"本章所讲的组合"与"框架第三根支柱"——hooks——之间的关系,hooks 是本书下一部分的主题。本章所教的"skill + subagent 组合",发生于 agent loop 内部;skill 与 subagent 都在语言模型的概率性空间里运作,它们塑造模型的行为,但并不对它施加硬

性约束。Hooks 则是另一回事——它位于 loop 之外,会对 loop 施加确定性约束。本书一路要带读者搭建出来的那块"完整工程化的 Claude Code 环境",靠的是三根支柱协同工作:skill

providing capability, subagents providing specialization, hooks providing the deterministic edges that make the whole environment safe and reliable to depend on. The reader who has finished this chapter has half of the architecture in place. The next part of the book completes it.

提供"能力",subagent 提供"专门化",hooks 则提供"确定性边缘"——正是它让整块环境可以放心依赖、安全可信。读到这里,读者手里的架构已经搭好了一半;下一部分书,把它补完。

A short observation is worth recording about the experience of working in a fully composed environment, although the reader will only verify the observation for herself once she has built one. Developers report that the felt difference between Claude Code without a configured environment and Claude Code with a thoughtful skills-plus-subagents composition is larger than they expected. The unconfigured tool feels capable but generic. The configured tool feels specific to the developer's work in a way that is initially surprising. Tasks that used to require explicit briefing now run themselves. Reviews that used to take ten minutes finish in three. The output that used to need editing for tone and convention arrives already in the right voice. The whole working day reshapes itself around the new floor of capability, and the developer's attention rises to fill the new space the environment has created. This is the practical payoff of the composition pattern, and it is the payoff that justifies the investment of time in building skill and subagent libraries thoughtfully rather than casually.

关于"在一块完全组合好的环境里工作"的体验,有一点观察值得先记下来——但读者要等自己真的搭出一套,才能亲自验证。开发者普遍反映:"未做配置的 Claude Code"和"做了深思熟虑的 skill+subagent 组合的 Claude Code"之间的体感差距,比他们预想的要大。未配置的工具"有能耐,但泛";配置过的工具"贴着开发者的具体工作而来",那份"贴",一开始甚至有点令人惊讶。以前需要专门简报才能跑起来的任务,如今自己就跑完了;以前要花十分钟的评审,现在三分钟就收;以前总要回头改语气和规约的产出,如今一上来就是对的噪音。整个工作日,会围绕这块"新的能力底线"重新排布,开发者的注意力随之向上抬,去填满环境新腾出来的空间。这就是组合模式在实务层面的回报——也是"花时间认真搭一套 skill 库与 subagent 库"这件事之所以值得的根本原因。

Part Three ends here. The reader has, over five chapters, learned the architecture of subagents, mastered the design of single specialists, internalized the orchestrator pattern that turns sequential work into parallel work, installed a library of eight working specialists, and seen how skills and subagents compose to produce the deeper architectural power of the system. The second pillar of the framework is now in place. The third pillar, hooks, is the subject of the next part of the book. Hooks are how the probabilistic system gets wrapped in deterministic edges, how the developer ensures that the things that must happen always happen, and how the boundaries of the engineered environment become unbreakable at the points where breakage would matter most. That work begins in the chapter ahead.

第三部分至此收束。读者用五章的时间,学会了 subagent 的架构、掌握了单个专家的设计、把"把串行变并行"的编排器模式内化下来,装好了一个含八位"能干活"的专家的库,也亲眼见到了 skill 与 subagent 是如何组合成系统更深层架构力量的。框架的第二根支柱,至此就位。第三根支柱——hooks——是本书下一部分的主题:它正是"用确定性的边缘去包裹一个概率性的系统"的那条路径,正是"必须发生的事永远会发生"那条工程化保障,也正是"工程化环境的边界,在最不该破的地方变得不可破"那条具体落地方式。这件事,由下一章开始做。

Before turning the page, a short closing thought. Each part of this book has built one layer of the larger architecture. Part One established the mental model. Part Two installed the first pillar. Part Three has installed the second pillar and shown how the first two pillars compose. The

pattern of work that the book asks of the reader is, in this respect, accumulating. Each part builds on what came before, and each successive layer makes the

翻页之前,还有一句简短的收束语。本书的每一部分,都搭建大架构中的一层:第一部分搭起心智模型;第二部分装上第一根支柱;第三部分装上第二根支柱,并展示了前两根如何组合。从这个意义上,本书要求读者投入的工作,是"累积"的——每一部分都建在前一部分之上,而后续每一层,都会让前一层

previous layers more powerful by giving them a richer environment to operate in. The hooks that the next part introduces will, in turn, make the skills and subagents already installed more reliable, because the deterministic guarantees that hooks provide will protect the probabilistic systems below them from the failure modes those probabilistic systems cannot prevent on their own. The architecture is genuinely hierarchical, and the experience of building the upper layers is, over time, the experience of giving the lower layers a sturdier foundation to stand on.

变得更强,因为它给了它们一个更富的环境去运行。下一部分要引入的 hook,反过来又会让已经装好的 skill 与 subagent 更加可靠——因为 hook 所提供的那些"确定性保证",会把下层那些"概率性系统"无法独自防住的失效模式,在它发生之前就拦下来。整套架构是真正分层级的;而搭上层这件事,日积月累,就是把下层的底座越筑越稳的那份体验。